BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF MECHANICAL ENGINEERING

FAKULTA STROJNÍHO INŽENÝRSTVÍ

INSTITUTE OF SOLID MECHANICS, MECHATRONICS AND BIOMECHANICS

ÚSTAV MECHANIKY TĚLES, MECHATRONIKY A BIOMECHANIKY

EMBEDDED IMPLEMENTATION OF AN ATTENDANCE SYSTEM FOR INTEGRATION INTO AN ENTERPRISE APPLICATION

EMBEDDED ŘEŠENÍ DOCHÁZKOVÉHO SYSTÉMU PRO INTEGRACI DO PODNIKOVÉ APLIKACE

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR — Jakub Hloušek

AUTOR PRÁCE

SUPERVISOR — Ing. Michal Bastl, Ph.D.

VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF MECHANICAL ENGINEERING

Bachelor's Thesis Assignment

Institute:	Institute of Solid Mechanics, Mechatronics and Biomechanics
Student:	**Jakub Hloušek**
Degree programm:	Mechatronics
Branch:	no specialisation
Supervisor:	**Ing. Michal Bastl, Ph.D.**
Academic year:	2025/26

As provided for by the Act No. 111/98 Coll. on higher education institutions and the BUT Study and Examination Regulations, the director of the Institute hereby assigns the following topic of Bachelor's Thesis:

Embedded Implementation of an Attendance System for Integration into an Enterprise Application

Brief Description:

The student will design and implement an attendance system based on an embedded platform, integrated with the company's internal software for work time recording. The system will use identification methods (such as RFID, NFC, or PIN code) to record employee arrivals and departures. The recorded data will be processed by the designed unit and transmitted to the company software through an appropriate communication interface. The thesis will include the design of the hardware and software architecture of the system, implementation of a functional prototype, and verification of its functionality in a model office environment.

Bachelor's Thesis goals:

1. Conduct a review of available hardware and software solutions for attendance systems and analyze possibilities for their integration with enterprise systems.
2. Design the architecture of the attendance system according to specific requirements for integration into the company's software.
3. Implement a prototype including an identification module (RFID/NFC/PIN) and a communication interface with the company's software.
4. Verification of the functionality of the solution in a model office environment and evaluation of its potential for practical deployment.

Recommended bibliography:

[1] VALÁŠEK, Michael. Mechatronika. Praha: České vysoké učení technické, 1995. ISBN 80-01-01276-X.

[2] HEROUT, Pavel. Učebnice jazyka C. 1. vydání. Brno: Computer Press, 2004. ISBN 80-251-0452-3

[3] MALÝ, Martin. ESP32 prakticky : od základních obvodů k pokročilým aplikacím. 1. vydání. Praha : CZ.NIC, z.s.p.o., 2024. 525 s. ISBN 978-80-88168-79-9.

Deadline for submission Bachelor's Thesis is given by the Schedule of the Academic year 2025/26

In Brno,

L. S.

prof. Ing. Jindřich Petruška, CSc. Director of the Institute	doc. Ing. Jiří Hlinka, Ph.D. FME dean

Abstrakt

Práce představuje návrh, implementaci a laboratorní ověření pasivního systému evidence docházky zaměstnanců, který nevyžaduje žádnou aktivní činnost ze strany uživatele. Dvě embedded zařízení umístěná na protilehlých stranách dveřního průchodu odesílají surová data z UHF RFID čteček na centrální server, který určuje směr průchodu a předává údaje o docházce na firemní platformu prostřednictvím modulární integrační vrstvy navržené tak, aby byla rozšiřitelná o další systémy.

Práce zahrnuje návrh a výrobu hardwaru na míru, vývoj firmwaru a procesní pipeline na straně serveru s živou integrací s firemním softwarem Navigo3.

Laboratorní validace potvrdila, že každý detekovaný průchod byl správně klasifikován z hlediska směru u všech zaznamenaných průchodů. Bylo zjištěno, že spolehlivost detekce závisí do značné míry na způsobu nošení tagu, přičemž za určitých podmínek dochází k výraznému snížení nebo úplnému zamezení detekce. Systém nikdy nevydá nesprávné určení směru, ale za nepříznivých podmínek nemusí vydat žádný signál. Jde o laboratorně ověřený koncept s identifikovanými podmínkami pro spolehlivý provoz.

Summary

This thesis presents the design, implementation and laboratory validation of a passive employee attendance system requiring no deliberate action from the user. Two embedded units mounted on opposite sides of a doorway publish raw UHF RFID readings to a central server, which determines traversal direction and forwards attendance events to an enterprise platform through a modular integration layer designed to be extensible to other systems.

The work encompasses custom hardware design and manufacture, firmware development, and a server-side processing pipeline with live integration with software Navigo3.

Laboratory validation confirmed that every detected traversal was correctly classified for direction across all captured events. Detection reliability was found to depend substantially on tag carry method, with certain conditions resulting in significantly reduced or zero detection rates. The system never produces an incorrect direction call, but may produce no call under unfavourable conditions. It is characterised as a laboratory-validated proof of concept with identified conditions for reliable operation.

Klíčová slova

UHF RFID, pasivní RFID, ESP32, FreeRTOS, KiCad, docházkový systém, evidence docházky, portálový model, detekce směru průchodu, temporální centroid, embedded systém, FreeCAD, MQTT, Navigo3

Keywords

UHF RFID, passive RFID, ESP32, FreeRTOS, KiCad, attendance system, attendance tracking, portal model, direction detection, temporal centroid, embedded system, MQTT, FreeCAD, Navigo3

I would like to thank my supervisor, Ing. Michal Bastl, Ph.D., for his guidance and patience throughout this work, and for the freedom he gave me to pursue the directions that turned out to matter.

Jakub Hloušek

Generative AI tools

Anthropic's Claude and Claude Code - were used during the preparation of this thesis. The full declaration of the scope, purpose, and verification of their use is given in Appendix A.

Acknowledgement

I affirm that the presented master's thesis is my genuine work and that it was created with the support of the stated literature, under the supervision of my tutor.

Jakub Hloušek

Contents

List of Figures

List of Tables

1. Introduction

Reliable recording of employee arrivals and departures is a baseline operational requirement for most enterprises, supplying the inputs to payroll, project time allocation, and compliance reporting. The technical realisation of this requirement sits at the intersection of embedded systems, automatic identification (RFID, NFC, or knowledge-based codes), and enterprise human-resources software. This thesis is concerned specifically with that intersection: the design and prototype implementation of an embedded identification platform whose recorded events feed directly into a company information system.

Commercially dominant attendance solutions - PIN terminals, contact-presented HF RFID card readers, and biometric (facial or fingerprint) terminals - share a structural property: each requires the employee to perform a deliberate identification action at a fixed point. This introduces friction at the threshold, creates queues at high-traffic times, and remains vulnerable to so-called buddy-punching, in which one employee enters or presents identification on another's behalf. A genuinely passive, zero-interaction system - one that records arrival and departure without any action from the employee beyond walking through the doorway - would eliminate all three of these issues simultaneously. No commercial product in the segment currently occupies this niche.

The concrete need for such a system arises at Navigo Solutions s.r.o., a Brno-based software firm whose product Navigo3 is a software-as-a-service company information system used by project-based businesses for project management, finance, capacity planning, and human-resources work, including arrival and departure records and absence tracking [1]. Prior to this thesis, Navigo3 exposed only a rudimentary internal attendance API and offered no provision for external hardware integrations; attendance entries were made manually through the web interface. As part of the present work, this API has been extended into a form that external clients may use to act on behalf of Navigo3 users, making the system described here the first hardware integration into Navigo3 that is not a bespoke build for a single customer. The same extension admits further integrations beyond the one developed in this thesis.

Two technical requirements follow from the zero-interaction goal. The first is reliable passive identification at a range of two to three metres, through clothing, bags, and pockets - the tag must be readable wherever an employee normally carries credentials. The second is the recovery of traversal direction: confirming that an employee crossed the doorway is not enough, since arrival and departure must be distinguished without any physical gate or turnstile. Resolving both requirements within a single self-contained embedded device, deployable at an arbitrary doorway with only mains power and wireless network connectivity, defines the scope of the system to be designed.

Of the candidate technologies surveyed in Section 2, only passive UHF RFID at 860-960 MHz combines a read range of several metres with the absence of any required user action. The approach adopted on top of this technology is a portal model: two autonomous embedded units, jointly named Lighthouse, are mounted on opposite sides of the doorway. Each unit independently publishes raw timestamped RFID readings to a central server over MQTT; the server clusters readings from the pair and infers traversal

direction from their joint temporal and signal-strength structure. Direction inference is therefore an entirely server-side responsibility, leaving each Lighthouse stateless with respect to its counterpart and free to operate, cache locally, and recover from network loss on its own terms. Successful direction-detected events are forwarded to Navigo3 through a connector layer isolated behind a single integration interface, so the same core pipeline may be extended to other enterprise platforms without modification.

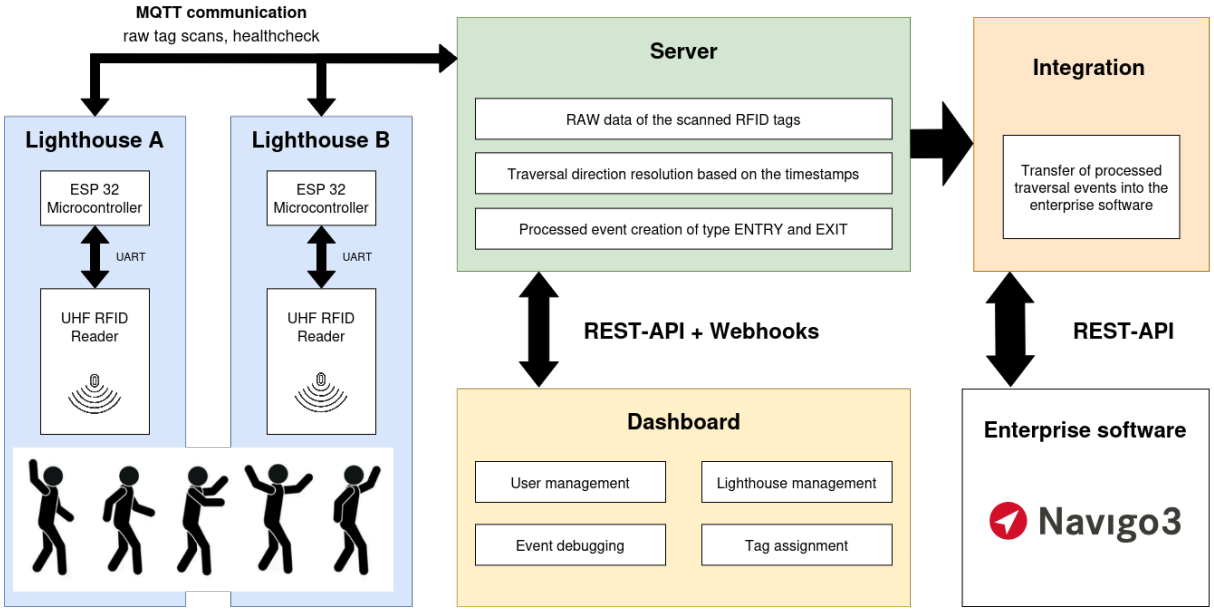

Figure 1: System architecture block diagram

The objectives of this thesis, as set out in the formal assignment, are:

- a review of existing hardware and software approaches to attendance recording, with attention to their integration into enterprise software environments;
- the design of a system architecture compatible with the integration constraints of the partner's information system;
- the construction of a functional prototype encompassing the identification subsystem and the data path to the partner's software; and
- verification of the prototype in a model office environment together with an assessment of its readiness for practical deployment.

The remainder of the thesis is organised as follows. Section 2 surveys identification technologies, established direction detection methods, candidate hardware platforms, and the software framework supporting the chosen microcontroller. Section 3 documents the system architecture, the hardware design across two board revisions, the firmware, the server pipeline, the Navigo3 integration layer, and the verification campaign. Section 4 summarises the outcomes and identifies directions for further development.

2. Research

2.1. Attendance System Technologies

Automated employee attendance tracking requires a reliable means of identifying a person at a physical boundary - a doorway, turnstile, or office entrance - without manual involvement from staff or administrators. Commercially available systems draw their identification mechanism from one of three categories: knowledge-based (PIN code), biometric (fingerprint, facial recognition), or token-based (a card or tag carried by the employee). The relevant question for this project is which of these can be made fully passive at a distance of two to three metres.

Knowledge-based systems require the employee to stop at a terminal and enter a code. Beyond the queueing this introduces, they are highly susceptible to so-called buddy-punching, where one employee enters another's code on their behalf. Knowledge-based identification is therefore unsuitable for any application aiming at hands-free operation.

Biometric systems can technically eliminate deliberate user action - a face is read while the employee walks past a camera - but introduce a considerably more serious obstacle. Biometric data processed for the purpose of uniquely identifying a person is classified as a special category of personal data under Article 9 of Regulation (EU) 2016/679 (GDPR), and its processing is by default prohibited absent one of the narrow legal bases enumerated in Article 9(2) [2]. Combined with a unit cost typically an order of magnitude above token-based readers, both factors disqualify biometrics for this project.

Token-based identification removes any requirement for the employee to stop or act; the employee simply carries a tag. The differentiating parameter between technologies in this category is read range. High-frequency RFID at 13.56 MHz - the technology behind ISO/IEC 14443 [3] and ISO/IEC 15693 [4] - operates in the near field and is restricted to roughly 0–10 cm, which forces the user to deliberately present the card; the experience is effectively equivalent to PIN entry. Ultra-high-frequency RFID at 860–960 MHz, governed by EPC Gen2 / ISO/IEC 18000-63 [5], operates in the far field and reaches 1–12 m with passive (battery-less) tags. A tag carried in a bag or pocket is detected at walking pace without any deliberate action by its carrier.

UHF RFID is therefore the only mature passive identification technology meeting the hands-free, 2–3 m range requirement, and is selected as the basis for this system.

Table 1: Identification technology comparison

Technology	Read range	User action	Sensitive data	Selected
PIN code	N/A	Yes	No	No
HF RFID (13.56 MHz)	0–10 cm	Yes	No	No
Biometric	Contact / 1 m	No	Yes	No
UHF RFID (860–960 MHz)	1–12 m	No	No	**Yes**

2.2. Direction Detection Methods

Distinguishing arrival from departure is a hard requirement for any attendance system; merely confirming that a tag was present at a location is not sufficient. A single reader provides only a presence event and cannot resolve direction, so direction information must be recovered from a second observable.

The standard approach is to mount two readers on opposite sides of the doorway and treat the pair as a portal. The temporal sequence of detections from the two readers carries the direction: detections that begin at the outside reader before the inside reader imply entry, and the reverse implies exit.

The simplest implementation is comparing the timestamps of the very first detection from each reader but is somewhat unreliable in practice. Passive UHF tags respond probabilistically and the radiation field in a real doorway is irregular due to multipath reflections; an early read from an RF null or a reflected wave can invert the apparent detection order. Oikawa demonstrates this failure mode experimentally on an RFID gate and proposes comparing the read-count-weighted temporal centroid of each reader's full detection group instead, which is far more robust to individual outlier reads [6].

A complementary signal is available in the received signal strength indicator (RSSI). As a tag traverses the portal, its RSSI at each reader rises while the tag approaches, peaks at the moment of closest approach, and falls as the tag moves away - a direct consequence of the inverse-square dependence of received power on distance described by the Friis transmission equation. The reader whose RSSI peaks first is therefore the reader the tag passed first; in the portal geometry this carries the same direction information as the temporal centroid by an entirely different physical mechanism. This principle is used as the primary direction cue in the RF-Access barrier-free access control system of Jie et al. [7].

Two algorithmic families therefore emerge from this literature: a temporal centroid algorithm following Oikawa's approach, and an RSSI-weighted centroid algorithm based on the Friis-derived peak-time argument. Both require the full set of raw timestamped RSSI readings from both readers - any per-device deduplication or summarisation discards the very signal the algorithms operate on. Detailed mathematical formulations of both families are given in Section 3.4.3.

2.3. Hardware Platforms and Embedded Architectures

The Lighthouse unit's hardware platform must support the full set of identification, network, and local-storage tasks within a single self-contained embedded device. The two principal sub-decisions are the choice of microcontroller and the choice of UHF RFID reader module.

2.3.1. Microcontroller Selection

The Lighthouse runs several concurrent tasks: UART communication with the RFID reader, WiFi connectivity and MQTT publishing, persistent local event storage, and management of GPIO peripherals. The microcontroller therefore needs integrated WiFi, at least one hardware UART, sufficient RAM to run a network stack alongside application logic, enough flash for firmware and an event-cache filesystem, a power profile compatible with battery-backed operation, a mature SDK and toolchain, and a low unit cost.

Single-board computers such as the Raspberry Pi satisfy the connectivity and processing requirements but are unsuitable on several other axes. They run a full Linux distribution with all of its boot, update, and management overhead; their power draw is substantially higher than a microcontroller's; and they typically depend on an SD card for persistent storage, with well-documented reliability problems in continuous embedded deployments. They are also significantly oversized for a single-peripheral embedded task. SBCs are therefore rejected.

Among microcontrollers with integrated WiFi, the ESP32 from Espressif Systems is the strongest match. It pairs a dual-core Xtensa LX6 CPU at up to 240 MHz with 520 KB of internal SRAM, integrated 2.4 GHz WiFi (802.11 b/g/n) and Bluetooth, and a rich peripheral set including multiple UARTs, I^2C, SPI, ADC, and hardware AES/SHA cryptographic accelerators [8]. The dual-core architecture is particularly relevant for this application: the WiFi/network stack can be pinned to Core 0 while the application logic runs on Core 1, eliminating cross-task interference between time-critical RFID handling and the inherently non-deterministic behaviour of a wireless network stack [9].

The ESP-IDF framework, Espressif's official SDK, integrates every component this application requires - a FreeRTOS kernel, the WiFi stack, an MQTT client, NVS, LittleFS, SNTP, and mbedTLS - all maintained by the silicon vendor [10]. Combined with extensive documentation, an active community, and a module unit cost of approximately 3–5 USD, the ESP32-WROOM-32 module is therefore the platform of choice for the Lighthouse unit; its application in the firmware is detailed in Section 3.3.

2.3.2. UHF RFID Reader Modules

UHF RFID readers are available across a wide cost and integration spectrum, from rack-mountable enterprise readers (Impinj R420 and similar) at hundreds of dollars to bare R-series chips intended for OEM integration. The relevant tier for this project is the embedded-integration tier: compact carrier modules that expose a UART command interface and can be controlled directly by a microcontroller. Within this tier, the selection criteria were a documented UART command protocol, 5 V supply compat-

ibility (matching the board's primary rail), and a unit cost suitable for small-scope prototyping.

Table 2: UHF RFID reader module candidates

Module	RF output	Supply	Interface	Price (approx.)
Impinj R420	+30 dBm	PoE	LLRP/Ethernet	$800+
ThingMagic M6e	+27 dBm	5 V	UART/USB	$200+
SparkFun M6E Nano	+27 dBm	3.3 V	UART	$60+
YPD-R300	**+25 dBm**	**5 V**	**UART**	**$15**

The YPD-R300 therefore best matches the project requirements. Its higher RF output relative to the R200 line supports the 2–3 m range requirement; its 5 V supply matches the board's main power rail directly; and its external SMA connector permits the antenna to be substituted during range testing, which is not possible with the integrated-antenna variant. The bare R-series chip option was rejected as it would require a custom RF front-end design, an unjustifiable scope expansion at the prototype stage. The Impinj R420 and the ThingMagic and SparkFun modules, while well-supported, were excluded on cost grounds; their per-unit price would consume a disproportionate share of the prototype budget for three units.

The 25 dBm RF output value cited in Table 2 reflects the module's actual hardware ceiling rather than the higher 33 dBm nominal value given on the manufacturer's datasheet; the relevant firmware-side handling of this discrepancy is discussed in Section 3.3.2.

2.3.3. Power Supply and Battery Considerations

The Lighthouse must operate from USB-C mains power while a lithium-ion cell provides backup autonomy. Both sources must be capable of powering the device simultaneously, with automatic and electrically safe arbitration between them so that connecting or disconnecting either source does not interrupt operation.

A single lithium-ion cell at 3.7 V nominal cannot directly supply either of the two regulated rails the device requires: the ESP32 needs a 3.3 V regulated supply, and the YPD-R300 RFID reader requires 5 V. A boost converter is therefore required to step the cell voltage up to 5 V, from which a low-dropout regulator derives the 3.3 V rail.

The constituent sub-problems - battery charging, cell protection (against overcurrent, overvoltage, and undervoltage), and arbitration between the USB and battery-derived 5 V rails via a Schottky-diode OR are well-established in embedded design practice, and dedicated single-function ICs exist for each role. The specific selections and the assembled topology are described on page 16.

One characteristic of this particular load deserves mention here, as it constrains the entire power chain: the UHF reader draws significant peak current during active scan windows. The supply must sustain these transients without rail collapse - an undersized converter or insufficient bulk decoupling will cause the ESP32 to brown out and reset. Sizing of converters, bulk capacitance, and the connections between them must therefore be dimensioned for peak rather than average current.

2.3.4. Timekeeping Without a Hardware RTC

The ESP32-WROOM-32 module does not include a battery-backed real-time clock. The internal RTC counter on the SoC continues to run during deep sleep but loses its value on a cold boot [8, §9.3.6]. For an attendance system, every event must carry a timestamp traceable to real calendar time, so a boot-relative counter is not by itself sufficient.

The standard approach for ESP32 timekeeping is synchronisation over the network using the Simple Network Time Protocol (SNTPv4) [11]. ESP-IDF includes a built-in SNTP client that synchronises the SoC's RTC counter against one or more configured time servers; the residual error after synchronisation is well below one second, which is more than adequate for attendance event timestamping.

The limitation of an SNTP-only design is that calendar time is lost whenever the device powers off or loses network access. This limitation is mitigated at the firmware data-model level; the mechanism is described in Section 3.3.3. A hardware RTC IC with coin-cell backup would eliminate the underlying limitation entirely and is identified as the primary hardware improvement for a future board revision.

2.4. Communication Protocols and Enterprise Integration

Two distinct communication paths exist in this system, with different requirements. The firmware-to-server path carries frequent small messages from a constrained embedded device over an unreliable wireless network. The server-to-enterprise path carries low-frequency, high-importance attendance records pushed from the server to an external HR system. The two paths are evaluated separately.

Table 3: Communication protocol comparison for firmware transport

Protocol	Suitability for firmware	Decision
HTTP/REST	Too heavy for frequent small publishes; no server push	Rejected for firmware; used server-to-enterprise
WebSocket	Reconnection logic burdens constrained clients	Rejected for firmware; used for dashboard
AMQP	No lightweight ESP-IDF client	Rejected
MQTT	**Designed for constrained devices on unreliable networks**	**Selected**

For the firmware-to-server path, MQTT is selected. It was designed specifically for constrained devices communicating over unreliable networks and provides tunable Quality-of-Service levels, automatic reconnection in the client library, and a Last Will and Testament (LWT) mechanism that publishes a broker-generated notification on unexpected disconnection [12]. The application of these features in the Lighthouse firmware is described in Section 3.3.4.

For the server-to-enterprise path, REST over HTTP is appropriate. Attendance records are created once per traversal event - a low-frequency, high-importance flow that

is well served by stateless idempotent HTTP endpoints, which are also universally supported by enterprise software. The integration target is Navigo3, the HR and project-management platform developed by Navigo Solutions s.r.o.; its REST API is built on the open-source `dry-api` framework, a typed JSON-over-HTTP transport [13], [14]. The platform's attendance-recording endpoints were extended in release 2026.03 with parametrised `start/stop` overloads, developed in conjunction with this thesis; the connector implementation is described in Section 3.4.5.

2.5. Embedded Software Frameworks

The ESP32 platform is supported by ESP-IDF (Espressif IoT Development Framework), the official vendor SDK. It bundles all of the components this project requires and avoids the fragmentation of assembling a firmware stack from independent libraries [9], [10].

FreeRTOS, integrated into ESP-IDF, is a preemptive real-time kernel providing tasks, queues, semaphores, and event groups [15]. Its dual-core scheduling model maps directly onto the ESP32′s architecture, with the WiFi stack assigned to Core 0 and application tasks to Core 1; the arrangement is detailed in Section 3.3.

NVS (Non-Volatile Storage), an ESP-IDF component, exposes a key-value store backed by an internal flash partition with transparent wear-levelling [10]. Its use for credential storage and persistent device state is described in Section 3.3.1.

LittleFS, a wear-levelling filesystem designed for NOR flash, is included as an ESP-IDF component [16]. It was selected over SPIFFS - the older ESP-IDF alternative - for its crash resilience and directory support; its role in the offline event cache is described in Section 3.3.4-B.

mbedTLS, also part of ESP-IDF, provides cryptographic primitives including AES-128, backed by the ESP32′s hardware AES accelerator [8, §14]. Its use for NVS credential encryption is described in Section 3.3.1-A.

For initial WiFi credential entry, the ESP-IDF `wifi_provisioning` component (BLE-based) was evaluated and rejected because it requires a companion mobile application, undermining the goal of a self-contained device with no auxiliary tooling on the employee or installer side. A custom captive-portal solution was implemented instead; the implementation is described in Section 3.3.1.

Table 4: ESP-IDF components used and their role

Component	Role
FreeRTOS	Task scheduling, inter-task queues, event groups
esp_mqtt	MQTT client - QoS 1/2, LWT, automatic reconnect
nvs_flash	Persistent key-value storage (credentials, config, state)
esp_littlefs	Offline event cache filesystem
esp_sntp	NTP time synchronisation
mbedTLS (AES)	Credential encryption via hardware AES accelerator
esp_wifi	STA + AP mode, captive portal provisioning

3. Implementation and Results

3.1. System Architecture

The Lighthouse attendance system is built around a portal model: two physically separate detection units are deployed on opposite sides of a doorway, with one unit designated as OUTSIDE and the other as INSIDE. These two units collectively form a single detection portal. When a person carrying a passive UHF RFID tag crosses the threshold, both units detect the tag in a temporal sequence that encodes the direction of movement, consistent with the dual-reader portal model established in Section 2.2. The fundamental architectural principle is that direction inference is a server-side responsibility - each Lighthouse unit operates autonomously, publishing only raw timestamped RSSI measurements over MQTT, with no knowledge of its paired counterpart and no attempt to determine direction locally. This keeps the embedded firmware thin, power-efficient, and focused on reliable tag detection and data transmission, while the detection algorithms remain centrally maintainable on the server.

Each Lighthouse unit is an embedded device built around the ESP32-WROOM-32 microcontroller communicating with a YPD-R300 UHF RFID reader module over UART (per ESP32 TRM Section 7; YPD-R300 Protocol Section 1.2). The unit includes onboard power management (USB-C input and lithium-ion battery backup), local offline event caching to a LittleFS partition on the ESP32′s flash memory, and WiFi/MQTT connectivity for scan data upload and health telemetry reporting. Four status LEDs provide visual feedback on WiFi connection state, MQTT broker connectivity, active RFID scanning, and tag detection events. Two buttons allow manual control of scan mode and WiFi provisioning entry, and an AM312 PIR motion sensor triggers automatic RFID scan windows when movement is detected near the portal.

Lighthouse units are logically paired on the server into a group, which becomes the unit of direction detection. Raw scans are clustered and processed per (tag EPC, group) pair. A Lighthouse can belong to at most one group at any given time; scans from ungrouped Lighthouses are orphaned by the event processing pipeline after a configurable timeout. Each group has independently tunable parameters: `activityTimeoutMs` (default 4 000 ms) defines how long after the last scan a cluster is considered closed and ready for processing, while `orphanTimeoutMs` (default 8 000 ms) determines when scans from misconfigured or unpaired devices are discarded.

The server is a single BunJS process that hosts an embedded Aedes MQTT broker, a PostgreSQL-backed REST API built with the Hono framework, and a WebSocket gateway for real-time dashboard updates. It also serves the SolidJS web dashboard as a static build from the same process. The server's responsibilities include: ingesting raw scan batches from MQTT and persisting them to the `raw_scans` table; running the EventSweeper background poller that clusters unprocessed scans and invokes both direction detection algorithms; managing user accounts and tag-to-user assignments; and pushing successfully processed events to the Navigo3 REST API with a retry-on-failure mechanism. The Navigo3 integration is isolated behind a connector interface controlled by environment variables, allowing the system to be extended to other enterprise platforms without modifications to the core event processing pipeline.

Table 5: System components and their responsibilities

Component	Technology	Responsibility
Lighthouse units	• ESP32-WROOM-32 • YPD-R300	• Passive UHF RFID detection • Raw scan publish via MQTT
Server	• BunJS + Hono • Aedes MQTT broker • PostgreSQL • Chrony NTP server	• Scan intake, direction detection • User management and API • Embedded MQTT broker • NTP time reference
Dashboard	• SolidJS SPA	• Device management • Event monitoring • User and tag setup
Navigo3 integration	• REST connector • Background retry poller	• Translates processed events into attendance records in Navigo3

The end-to-end event lifecycle from tag detection to attendance record creation proceeds as follows. An IR motion sensor detects movement near the portal, triggering the firmware to open a 5-second RFID scan window. The YPD-R300 reader runs continuous real-time inventory during this window, with detections batched on the firmware side and published to the MQTT topic `lighthouse/{id}/scans` as timestamped RSSI arrays. The server's scan handler ingests each batch, validates individual elements, and persists valid scans to the `raw_scans` table while preserving the `timeBasis` field that indicates timestamp quality. The EventSweeper background poller runs every 2 seconds, clustering unprocessed scans by (EPC, group). Once a cluster is considered closed (no new scans for `activityTimeoutMs`), both direction detection algorithms execute and each produces an independent row in the `processed_events` table. The processed event is immediately pushed to Navigo3 via its `start` or `stop` endpoint depending on the detected direction. If the push fails, a separate background retry sweep picks it up within the configured `NAVIGO3_RETRY_INTERVAL_MS`. If connectivity is lost before the firmware can publish to MQTT, scans are cached to a LittleFS ring buffer on the ESP32's flash and replayed in order on reconnection with QoS 2 for delivery guarantees.

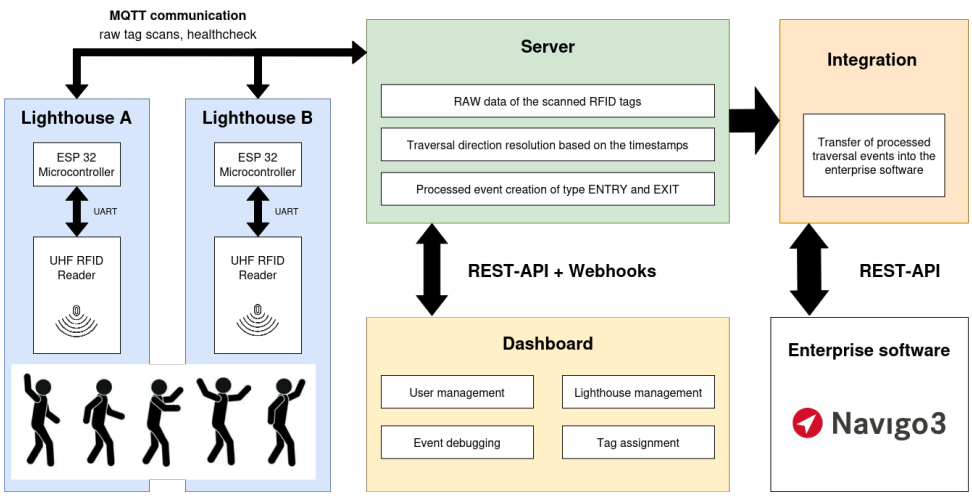

Figure 2: System architecture block diagram

3.2. Hardware Design

3.2.1. Board v1 - Breadboard Prototype

The first functional prototype was assembled on a breadboard using four off-the-shelf breakout modules: an ESP32-WROOM-32 development board (38-pin variant), a YPD-R300 UHF RFID reader module on its carrier board, an SX1308 step-up DC-DC converter module, and a TP4056 lithium battery charger module with integrated DW01HA protection IC and FS8205A dual MOSFET. None of these modules shipped with complete engineering documentation beyond basic pinout labels; each was therefore reverse-engineered into a schematic capturing component values, internal connections, and undocumented design decisions. These schematics became the baseline for the Board v2 custom PCB, whose goal was to reproduce and improve upon the combined functionality of the four modules on a single integrated board.

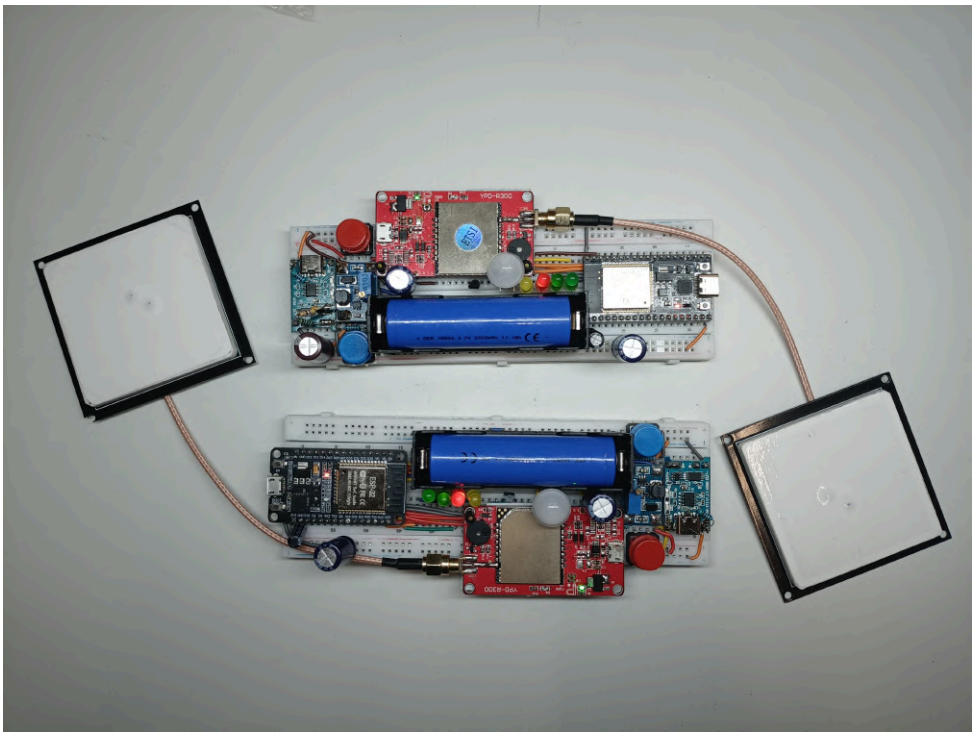

Figure 3: Photograph of the two Board v1 breadboard prototypes

Component selection and concurrency architecture were validated on the breadboard. The ESP32-WROOM-32 proved capable of running the WiFi network stack, MQTT client, and UART-driven RFID operations concurrently using the dual-core Xtensa LX6 architecture, with the WiFi stack pinned to Core 0 and application logic running on Core 1 (per ESP32 TRM Section 1.1). The YPD-R300 reader communicates with the ESP32 via UART at 115200 baud (per YPD-R300 Protocol Section 1.2). GPIO assignments for UART TX/RX, status LEDs, buttons, PIR motion sensor input, and the BC337 NPN transistor-based RFID power switch were established during breadboard testing and carried forward unchanged to Board v2. Power delivery on the breadboard used point-to-point wiring between the four modules, which introduced uncontrolled trace impedance and measurable voltage drops under pulsed RFID load, motivating the decision to move to a purpose-designed PCB with dedicated decoupling capacitance positioned close to load switching points.

Table 6: Reverse-engineered modules and their Board v2 disposition

Module	Key components identified	Disposition in Board v2
ESP32	• AMS1117-3.3 LDO • CP2102 USB-UART • EN/BOOT buttons	• TS1117B substituted for AMS1117 • CP2102 removed
YPD-R300	• R300 module • SMA connector • Decoupling capacitors	• Circuit reproduced accurately
SX1308 boost	• SX1308 IC • Schottky diode • Feedback divider	• Circuit reproduced accurately
TP4056 charger	• TP4056 IC • DW01HA protection • FS8205A dual MOSFET	• Circuit reproduced accurately

3.2.2. Board v2 - Custom PCB

Board v2 is a single two-layer PCB integrating all functionality of the four breadboard modules plus additional circuitry for power switching, fuse protection, battery voltage monitoring, and consolidated USB-C connectivity. The board was designed in KiCad with the schematic split into four hierarchical sheets (see Appendix C): a top-level Lighthouse sheet, a Charger submodule sheet, a Step-up DC/DC converter sheet, and a UHF RFID reader sheet. This hierarchical structure mirrors the modular nature of the breadboard prototype and keeps each functional block's schematic content manageable and self-contained. The CP2102 USB-UART bridge present on the original ESP32 development board was intentionally omitted from Board v2 to reduce component cost and simplify SMD assembly. In its place, UART TX, RX, and GND are broken out to a 3-pin header, allowing firmware debugging and flashing via an external USB-UART adapter. The reader is assumed to have the complete schematic sheets from Appendix C available for side-by-side reference; the following subsections describe key design decisions and deviations from the baseline breadboard design rather than replicating full schematic content.

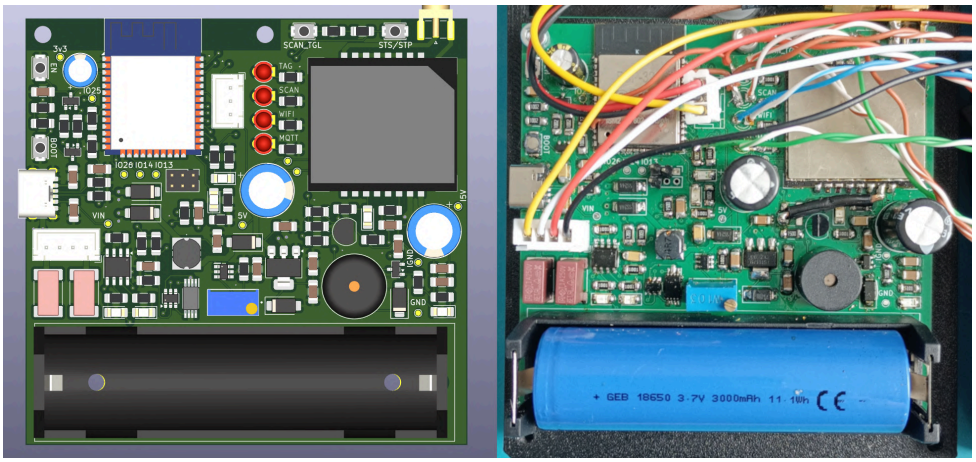

Figure 4: Comparison between the KiCad 3D render and the assembled Board v2 PCB; top view showing component placement

GPIO Assignments
The Board v2 schematic defines all GPIO connections between the ESP32-WROOM-32 module and peripherals. These assignments were validated during breadboard testing and are documented in the top-level Lighthouse schematic sheet (Appendix C, Sheet 1).

Table 7: ESP32 GPIO assignments on Board v2

GPIO	Peripheral	Function
4	LED1 (green)	WiFi + MQTT combined status indicator
21	LED2 (green)	IR mode / AP provisioning indicator
26	LED3 (red)	Active RFID scan indicator
25	LED4 (yellow)	Tag detection flash / battery status
22	BUTTON1	Scan mode control
23	BUTTON2	Status message / WiFi setup trigger
19	IR_SENSOR	AM312 PIR motion sensor input
16	R300 UART RX	UART receive from YPD-R300
17	R300 UART TX	UART transmit to YPD-R300
5	RFID_POWER_SWITCH	BC337 base drive for R300 power control
33	BATTERY_SENSE	ADC1_CH5 for battery voltage monitoring

GPIO5 is a strapping pin on the ESP32 (per ESP32 Datasheet Section 2.3) and must be held high during boot to select SPI boot mode. The Board v2 schematic includes a 10 kΩ pull-up resistor on GPIO5 to ensure reliable boot behaviour while still allowing it to function as a general-purpose output for the BC337 base drive after boot completes. GPIO33 was selected for battery voltage monitoring because it is connected to ADC1_CH5, which remains available for use during active WiFi operation; ADC2 channels share hardware with the WiFi RF subsystem and cannot be sampled reliably while WiFi is active (per ESP32 TRM Section 31.4.2).

USB-C Connector
The breadboard prototype used two USB connectors: one for power and one for the CP2102 UART bridge. Board v2 consolidates power input into a single USB-C connector carrying VBUS and GND only; the UART debug interface moves to the dedicated 3-pin header described above. USB-C requires 5.1 kΩ pull-down resistors on both CC1 and CC2 to declare the device as a power sink (per USB Type-C Cable and Connector Specification Section 4.5.1); Board v2 includes both. Without them the sink role is undeclared and a specification-compliant source will not supply VBUS.

Power Delivery Architecture
Board v2 implements a dual-input power architecture with automatic source selection (see Appendix C). The primary source is USB 5 V from the USB-C connector; the backup is a single-cell lithium-ion battery (nominal 3.7 V, operating range 3.2–4.2 V) stepped up to 5 V by an SX1308 boost converter. Each input path carries an SF-1206SP100-2 slow-blow fuse (1 A, 63 VDC, 1206 package), selected to tolerate the YPD-R300′s power-on inrush without spurious tripping. Downstream of the fuses, a DPDT slide switch (SLW-1678105-6A-N-D, 1 A / 12 VDC) switches both paths simultaneously for complete power isolation while keeping the fuses always in-circuit.

Two SS24A Schottky diodes (DO-214AC, 2 A / 40 V, forward drop 0.3–0.4 V) then form an OR junction that prevents backfeed between the two sources.

The 5 V rail downstream of the OR junction feeds a TS1117B-3.3 LDO (per TS1117B datasheet) producing the 3.3 V rail for the ESP32-WROOM-32 and peripheral logic. The YPD-R300 operates directly from the 5 V rail, switched via a BC337-25 low-side transistor (Section 3.2.2-D). When USB power is present, the TP4056 charger draws from VBUS to charge the cell, with the DW01HA protection IC and FS8205A dual MOSFET providing overcharge, overdischarge, and overcurrent protection (per TP4056 and DW01HA datasheets). On USB removal the system switches to battery power via the SX1308 boost converter without interruption.

RFID Reader Power Switching

The YPD-R300 draws approximately 380 mA at 5 V during active inventory (per YPD-R300 datasheet). To conserve power when the reader is idle, Board v2 switches its ground return through a BC337-25 NPN transistor (TO-92) configured as a low-side switch, controlled by ESP32 GPIO5. The BC337-25 was selected for its low saturation voltage ($V_{CE(sat)}$ 300 mV at 380 mA per BC337 datasheet) and 800 mA collector current rating. A 150 Ω base resistor supplies approximately 17 mA of base drive at GPIO5 high (3.3 V), forcing hard saturation - a forced beta of roughly 22 at the 380 mA operating point - which keeps $V_{CE(sat)}$, and therefore the switch's ground offset, small enough not to affect YPD-R300 operation.

Supply Stabilisation and Decoupling

The YPD-R300 reader draws current in sustained 18.8 ms RF transmission pulses occurring at 30–50 ms intervals during continuous inventory operations. This pulsed load profile requires bulk capacitance rather than ceramic-only decoupling to prevent supply rail droop that could cause brownout resets on the ESP32. Board v2 uses a combination of electrolytic bulk capacitors and ceramic bypass capacitors positioned close to each load.

Table 8: Decoupling capacitance placement

Location	Capacitance	Type / Notes
R300 5 V rail	470 µF + 100 nF	Electrolytic bulk + ceramic bypass
ESP32 3.3 V rail	100 µF + 100 nF	Electrolytic bulk + ceramic bypass
SX1308 output (5 V)	100 µF	Electrolytic; per SX1308 reference design
TS1117B input (5 V)	10 µF	Electrolytic; per TS1117B datasheet
TS1117B output (3.3 V)	22 µF	Electrolytic; per TS1117B datasheet

The 470 µF electrolytic capacitor on the R300 5 V rail was sized to limit voltage droop during the reader's 18.8 ms RF pulses to less than 200 mV, based on the measured current draw and acceptable ripple tolerance. Ceramic 100 nF bypass capacitors are placed immediately adjacent to the power pins of the R300 module and the ESP32-WROOM-32 module to suppress high-frequency switching noise. The SX1308 boost converter and TS1117B LDO capacitor values follow the respective datasheets' recommended application circuits.

3.2.3. Antenna and RF Considerations

UHF RFID operates in the 860–960 MHz range (ETSI band 865–868 MHz in Europe, FCC 902–928 MHz in North America). At these frequencies the integrity of the feed path between the YPD-R300 RF output and the antenna directly governs detection range; the R300 RF output is specified for 50 Ω impedance (per YPD-R300 datasheet Section 3.1; see also Section 2.3.2), and any mismatch reflects power away from the antenna. Board v2 routes the R300 RF output to a board-edge SMA coaxial receptacle via a 2 cm microstrip trace.

The trace width on Board v2 was not impedance-controlled during layout. Achieving 50 Ω characteristic impedance on a microstrip requires the trace width to be matched to the PCB substrate thickness, dielectric constant, and copper weight, none of which were explicitly calculated or verified for the chosen stack-up. The empirical consequence is visible in Table 19. The Blue unit, fitted with a 4 dBi antenna soldered directly to the R300 RF output pad, and the Yellow unit, fitted with a 5.5 dBi antenna routed through the SMA receptacle and the 2 cm trace, both achieve the same 3.0 m maximum reliable range. Vendor-stated ideal free-space ranges for the two antennas are 3.5 m and 4.8 m respectively [17], [18]; the Blue unit therefore achieves approximately 86% of its antenna's stated ideal, while Yellow achieves only 62%. Were Yellow's feed path as efficient as Blue's, the larger antenna's expected range would be approximately 4.1 m. The recoverable range loss attributable to the unmatched feed path is therefore estimated at 1 to 1.5 m, the lower bound coming from the proportional argument above and the upper bound allowing for additional loss in connector transitions not present on the direct-solder unit.

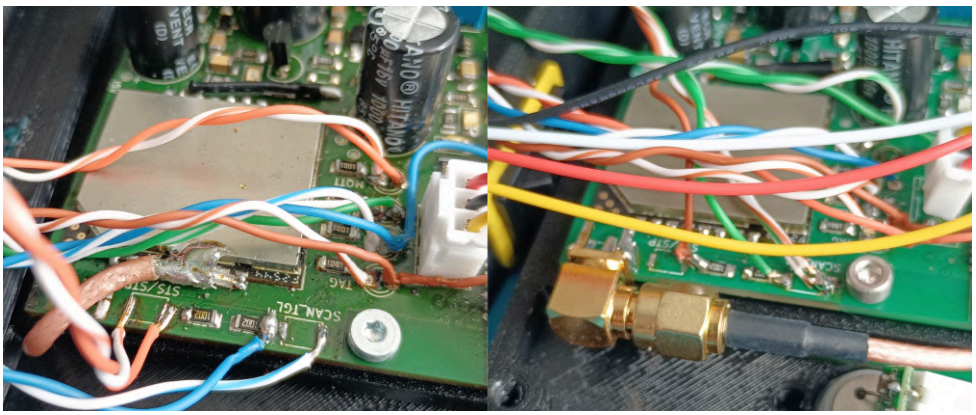

Figure 5: Antenna conenction comparison

A future board revision should reposition the SMA receptacle immediately adjacent to the R300 RF output pad to eliminate the trace entirely. Should a non-trivial trace remain unavoidable in a later layout, its geometry must be calculated for 50 Ω characteristic impedance against the chosen PCB stack-up before fabrication.

3.2.4. Enclosure

A prototype enclosure was designed in FreeCAD to house the Board v2 PCB, battery, PIR sensor, and antenna in a wall-mountable form factor suitable for doorway deployment. The design addresses several constraints imposed by the operational requirements of a passive detection system. The PIR sensor window must face the detection zone to trigger RFID scan windows when personnel approach the portal. The antenna must

be oriented toward the doorway with minimal physical obstruction to maintain the detection range validated during board testing. The USB-C port must remain accessible for charging and firmware updates without disassembling the enclosure. The four status LEDs must be visible to operators for diagnostic purposes, implemented via light pipes from the PCB-mounted LEDs to the enclosure front face. The two push buttons must remain accessible for manual scan mode control and WiFi provisioning entry.

Figure 6: Manufactured units - PIR sensor, antenna position, LED light pipes, and USB-C port access

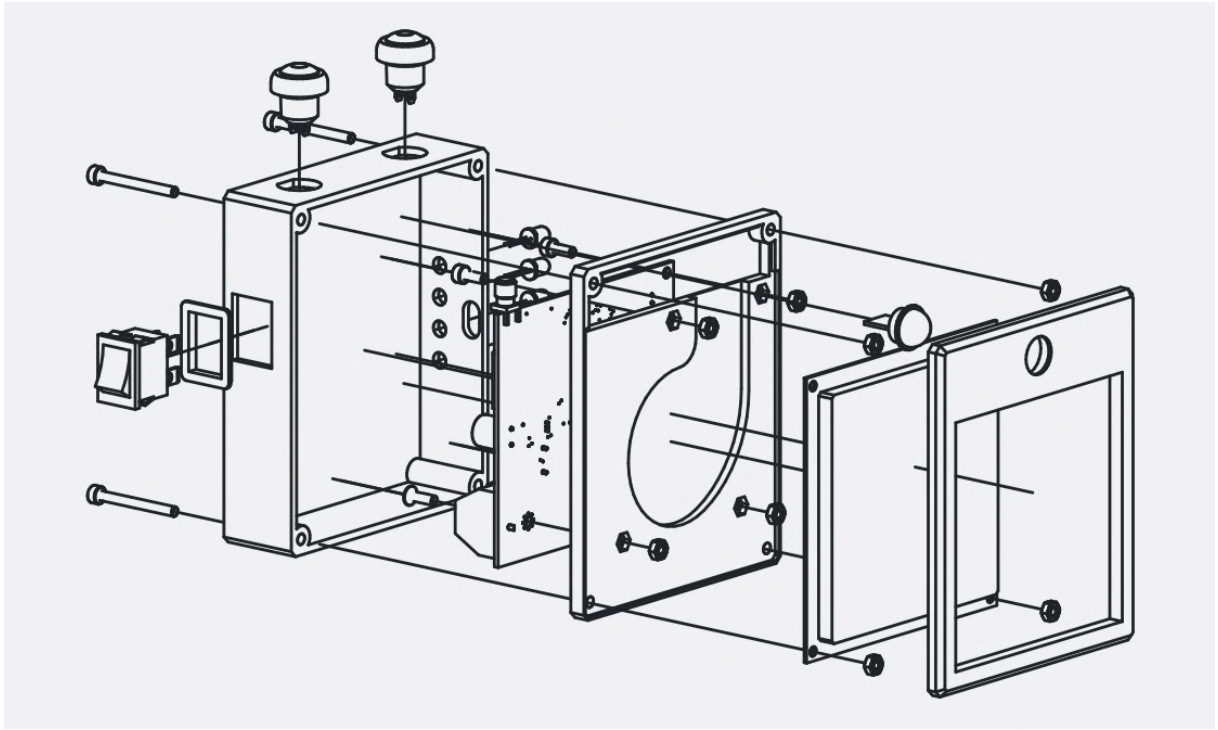

Figure 7: CAD model screenshot - exploded or open view showing internal component placement: PCB, battery, antenna mounting

Three enclosures were manufactured via 3D printing and assembled, one for each fabricated Board v2 unit currently in operation. Full technical drawings of the enclosure including dimensioned orthographic projections and section views are provided in Appendix D.

3.3. Firmware

The Lighthouse firmware is written in C using ESP-IDF v5.x with FreeRTOS as the underlying real-time operating system. The entire application runs on a single ESP32-WROOM-32 module, taking advantage of the dual-core Xtensa LX6 architecture to separate concerns between networking and application logic. By default, ESP-IDF pins the WiFi stack to Core 0, leaving Core 1 available for application tasks including RFID reader communication, offline event caching, and the main control loop (per ESP32 TRM Section 1.1). This isolation ensures that WiFi stack operations do not interfere with the timing-sensitive UART communication with the YPD-R300 reader module.

The firmware binary is identical across all Lighthouse units deployed in the field - no unit-specific configuration is compiled in at build time. All per-device settings are configured at runtime via the provisioning system and stored persistently in the ESP32′s NVS. This design allows a single firmware image to be flashed to multiple devices during manufacturing or field deployment, with each device then individually configured via the captive portal interface without requiring a recompile or reflash.

3.3.1. System Initialization and WiFi Provisioning

On boot the firmware initialises its subsystems in a fixed sequence, shown in Figure 8. Two branch points alter this path. If a provisioning flag is set in RTC-backed memory, the device enters AP-mode provisioning instead of connecting (Section 3.3.1-A); if WiFi cannot be reached, the device proceeds in offline mode, caching scans locally until connectivity is restored (Section 3.3.4-B). On the very first boot, with no credentials in NVS, the firmware halts after GPIO setup and waits for the user to trigger provisioning.

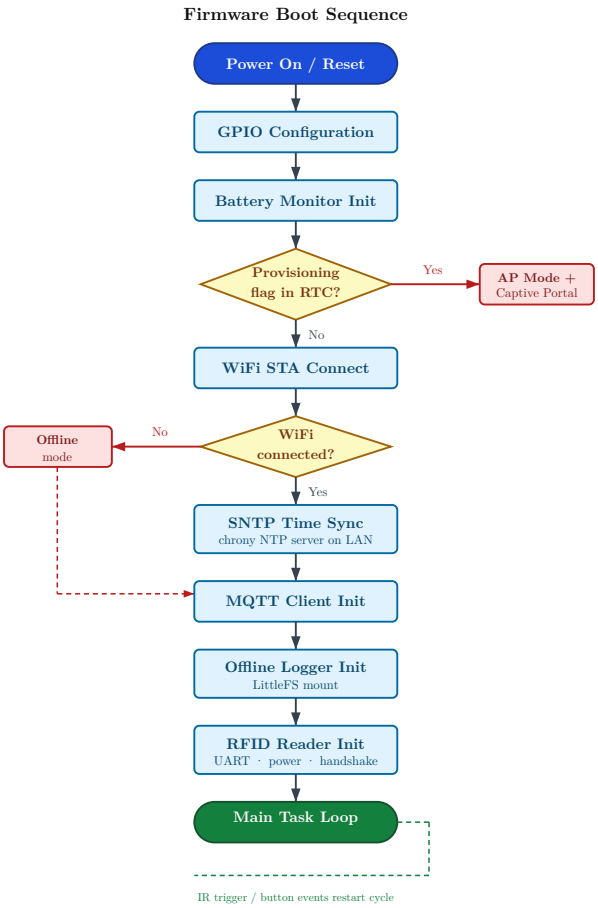

Figure 8: Provisioning state machine diagram

Provisioning Flow

Provisioning is triggered by a 5-second hold of BUTTON2, which sets a flag in RTC-backed memory and calls `esp_restart()`; the reboot guarantees AP mode starts from a clean state with no residual tasks or connections from normal operation. On the subsequent boot the firmware detects the flag during early initialisation and enters Access Point

mode. In AP mode, the ESP32 broadcasts an open WiFi network with the SSID "Lighthouse-Setup" and starts an HTTP server listening on port 80. The server serves a minimal HTML configuration form stored in the firmware's SPIFFS virtual filesystem partition. Concurrently, a lightweight DNS server is started that responds to all DNS queries. This captive-portal approach was adopted in preference to the ESP-IDF BLE-based `wifi_provisioning` component evaluated in Section 2.5. The DNS hijacking enables captive portal detection on iOS, Android, Windows, and macOS; when a user's device connects to the "Lighthouse-Setup" network, the operating system automatically detects the captive portal and opens a system browser window to the provisioning page without requiring the user to manually navigate to an IP address.

The provisioning form collects four fields: WiFi SSID, WiFi password, MQTT broker IP address, and MQTT broker port. Client-side JavaScript validates the input format before allowing submission. When the user submits the form, the device attempts to connect to the specified WiFi network in STA mode while keeping the AP active. If the

connection succeeds, the firmware then attempts to connect to the MQTT broker at the provided IP and port to verify end-to-end connectivity. The results of both tests are reported back to the browser. The device then reboots when the user clicks the restart button on the page letting the device to begin normal operation.

Credentials are encrypted before being written to NVS. The firmware uses AES-128 in ECB mode via the mbedTLS library utilizing ESP32's hardware AES accelerator to perform encryption and decryption (per ESP32 TRM Section 14). The encryption key is a device-specific 16-byte constant defined at compile time in a configuration header; in a production deployment, this key would be unique per device and generated during the firmware build process. The SSID is zero-padded to 32 bytes (two AES blocks) and the password is zero-padded to 64 bytes (four AES blocks) before encryption. The encrypted blobs are stored in a dedicated NVS namespace within the `nvs_settings` partition, separate from other configuration data to reduce the risk of corruption during writes. On subsequent boots, the credentials are loaded from NVS, decrypted in memory, and used to automatically connect to the provisioned WiFi network without user intervention.

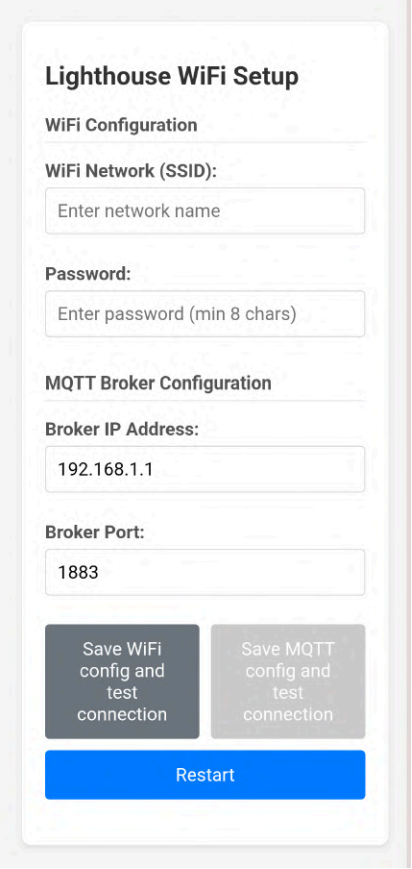

Figure 9: Screenshot of the provisioning web form as rendered on a mobile device

3.3.2. UHF RFID Scan Control

The YPD-R300 UHF RFID reader module communicates with the ESP32 via UART at 115 200 baud, configured on GPIO16 (RX) and GPIO17 (TX) using UART2 (per R300 Protocol V2.2 Section 1.2; ESP32 TRM Section 7.8). RFID scanning is performed using the real-time inventory command `0x89` (`cmd_name_real_time_inventory`) with channel parameter `0xFF` to enable full-spectrum frequency hopping and minimize round duration, and a 10 ms inter-round delay configured in firmware (per R300 Protocol

Section 2.2.8, page 27). In real-time inventory mode, the reader streams tag detection packets to the ESP32 as they occur during each RF transmission round, rather than buffering them internally and sending a consolidated response at the end. Each packet contains the tag's Electronic Product Code (EPC) identifier, received signal strength indicator (RSSI) in dBm, frequency channel and antenna ID, and the ISO 18000-6C protocol control (PC) word. Under typical operating conditions with a single tag in range, a 5-second scan window produces approximately 60 individual tag detection packets - corresponding to roughly 12 RF rounds per second with an average of 5 detections per round.

IR-Triggered Scanning

The AM312 PIR (passive infrared) motion sensor mounted on GPIO19 detects movement in the doorway and triggers RFID scan bursts via a GPIO interrupt configured for rising edge detection with an IRAM-safe interrupt service routine (`IRAM_ATTR` attribute ensures the ISR code is loaded into internal RAM to avoid flash cache misses during interrupt execution). When the IR sensor fires, the firmware powers on the R300 module by driving the BC337-25 NPN transistor base via GPIO5, waits 200 ms for the reader's internal oscillator to stabilize, performs a UART handshake by sending a firmware version query command (`0x72`) to verify that the reader is responsive and correctly initialized, and then starts real-time inventory. The scan burst runs for a duration defined by `IR_SCAN_DURATION_MS`, defaulting to 5s. If additional IR triggers arrive while a scan burst is already active, the burst timer is restarted without interrupting the ongoing inventory operation - this extends the scan window when a person lingers near the doorway rather than passing directly through, ensuring that all tag detections during their presence are captured. When the burst timer expires with no further IR triggers received, the inventory command is stopped by sending a `0x28` stop command to the R300 module, and the module is powered off via the transistor switch to conserve battery power.

MQTT Batch Accumulator

Tag detections from the R300 are accumulated in a time-windowed buffer on the firmware side rather than published one MQTT message per detection. The buffer flushes whenever either of two conditions is met: elapsed time since the first buffered detection reaches `scan_batch_ms` (default 150 ms, configurable via NVS, range 50–2000 ms), or the buffer fills to its capacity of `MQTT_SCAN_BATCH_MAX_ENTRIES` (64 entries). On flush, the firmware publishes the accumulated detections as a single JSON array to topic `lighthouse/{id}/scans`. The live scan path uses QoS 1; QoS 2′s four-way handshake is reserved for the offline replay path (see Section 3.3.4-B), where exactly-once delivery prevents duplicate attendance records from re-delivered cached batches.

Transmit Power Configuration

The YPD-R300 variant used in this project caps the power amplifier at 25 dBm (cf. Section 2.3.2). The firmware clamps the configured transmit power to the 20–25 dBm range before issuing the `set_power` command (`0x76`, per R300 Protocol §2.1.7, p. 12), reads the response packet, and validates the module's success code `0x10`; values outside the supported range are rejected by the module with error code `0x48` ("output_power_out_of_range", per R300 Protocol §3, p. 39). The validated power level is logged on every successful configuration.

3.3.3. Timekeeping and Timestamp Quality

As established in Section 2.3.4, the ESP32 does not include a battery-backed hardware Real-Time Clock (RTC); the internal RTC timer runs from a 150 kHz oscillator with approximately 5% drift under typical operating conditions (per ESP32 Datasheet Section 3.3.4). After a power cycle or hard reset, the system clock starts from an undefined epoch and must be synchronized via SNTP before timestamps represent meaningful wall-clock time. To ensure that the server can correctly interpret every event timestamp regardless of when it was recorded relative to SNTP synchronization, the firmware implements a three-tier time quality model in which every scan event payload includes a `timeBasis` field that declares the trustworthiness of its accompanying timestamp.

Table 9: Time quality tiers

Quality level	timeBasis value	Condition	Accuracy
Synced	`synced`	SNTP synchronisation completed successfully	Millisecond-level (network + NTP jitter)
Estimated	`estimated`	No SNTP sync yet, but last-known-good time recovered from NVS	Seconds to minutes (RTC drift 5% at 150 kHz)
Relative	`relative`	No NVS time available; first boot or NVS lost	Boot-relative milliseconds only; not a real wall-clock time

SNTP Synchronisation

The firmware uses the ESP-IDF SNTP client component in polling mode, configured to synchronize against a chrony NTP server running on the same host as the MQTT broker. On successful synchronization, an SNTP callback registered via `sntp_set_time_sync_notification_cb()` upgrades the time quality state from `TIME_QUALITY_NONE` or `TIME_QUALITY_ESTIMATED` to `TIME_QUALITY_SYNCED`, and persists both the current Unix timestamp (in seconds) and the ESP32 uptime counter value (in milliseconds from `esp_timer_get_time()`) to NVS under keys `last_unix_s` and `last_uptime_ms` in the `time_sync` namespace. This timestamp-uptime pair allows the firmware on the next boot to estimate wall-clock time even before SNTP synchronization completes, by loading the persisted values and recognizing that the current time must be at least `last_unix_s` seconds past the Unix epoch (it cannot be earlier, even if the device was powered off for an extended period). The timezone is set to Central European Time with automatic daylight saving transitions (`TZ=CET-1CEST,M3.5.0,M10.5.0/3`) via `setenv("TZ", ...)` and `tzset()` before SNTP initialization, ensuring that `localtime()` and `gmtime()` return correctly offset local time immediately upon successful synchronization.

Offline Degradation

If SNTP synchronization does not complete within a reasonable timeout (for example, if the NTP server is unreachable due to network configuration issues or server downtime), the firmware continues normal operation with `estimated` or `relative` timestamp quality rather than blocking indefinitely. Events logged during this degraded time state carry the `timeBasis` field set to `estimated` if a last-known-good time was recovered from NVS,

or `relative` if no NVS time is available (first boot after flash erase). The server-side scan ingestion handler reads the `timeBasis` field on every incoming scan and uses it to decide whether the scan is eligible for direction detection processing. Scans with `timeBasis` values other than `synced` are flagged and handled by orphan processing logic on the server, which either rejects them entirely or uses the MQTT message arrival timestamp (`received_at`) as a best-effort fallback for temporal ordering.

3.3.4. MQTT Communication and Offline Caching

Following the protocol selection in Section 2.4, the transport is offline-first: every scan event is written to a LittleFS ring buffer before any transmission attempt, and replayed from cache on reconnection. The live scan publish path uses QoS 1 to minimise handshake overhead during high-frequency scan windows; the offline replay path uses QoS 2 for exactly-once delivery guarantees, preventing duplicate attendance records from replayed batches.

MQTT Client

The firmware connects to the MQTT broker using the `esp_mqtt_client` component included with ESP-IDF, with connection parameters (broker IP address and port number) loaded from NVS where they were stored during the WiFi provisioning flow. The MQTT client ID is derived from the ESP32′s eFuse MAC address (a factory-programmed unique identifier burned into one-time-programmable memory per ESP32 TRM Section 4.4), ensuring that each device has a unique client ID to prevent broker-side connection conflicts when multiple Lighthouse units connect to the same broker. A Last Will and Testament (LWT) message is registered when the MQTT connection is established, configured to publish a disconnection notification to the topic `lighthouse/{id}/lwt` with QoS 1 and the retain flag set. This allows the server to detect unexpected disconnections (such as power loss or network failure) without relying on periodic keepalive polling; if the ESP32 disconnects without sending a graceful DISCONNECT packet, the broker automatically publishes the LWT message on the client's behalf.

Table 10: MQTT topics (prefixed `lighthouse/{id}/`) and QoS levels

Topic	Direction	QoS	Content
`scans`	Publish (live)	1	Batched JSON array of tag detections
`scans`	Publish (replay)	2	Replayed offline-cached events
`health`	Publish	1	Periodic health telemetry (system, RFID, battery)
`config`	Subscribe	1	Runtime configuration updates from server

Offline Event Caching

When the MQTT connection is unavailable - either because the broker is unreachable, the network link is down, or the device has not yet completed WiFi association - tag detection events are stored to a dedicated LittleFS partition on the ESP32′s internal flash memory. This partition is entirely separate from the SPIFFS partition used to store the WiFi provisioning HTML form; SPIFFS is a read-only filesystem image compiled into the firmware binary at build time using the `spiffs_create_partition_image()` CMake function, whereas LittleFS is a writable wear-leveling filesystem mounted at runtime and used exclusively for offline event storage. Events are written to a ring buffer file named `events.bin` stored in the root of the LittleFS mount point, with each event

entry protected by a CRC32 checksum to detect corruption from partial writes or flash wear. Write and read index pointers are persisted to two independent locations: RTC memory (a small region of SRAM that survives soft resets triggered by `esp_restart()` but is cleared on hard power-down) for fast recovery after watchdog resets or firmware updates, and NVS (Non-Volatile Storage) for recovery after power loss.

On MQTT reconnection, a background replay task (`offline_replay_task`) is spawned with lower priority than the main event logging task to ensure that real-time tag detections are not delayed by replay activity. The replay task reads cached events from the LittleFS ring buffer and publishes them to the same MQTT topic (`lighthouse/{id}/scans`) at a throttled rate of at most 10 events per second, with each replayed event carrying QoS 2 for exactly-once delivery guarantees. The firmware adds an `offline: true` boolean flag and a `replayTime` field containing the Unix timestamp at the moment of replay to each replayed event payload, allowing the server to distinguish replayed historical events from live real-time detections and to reconstruct the timeline accounting for the period during which the device was offline.

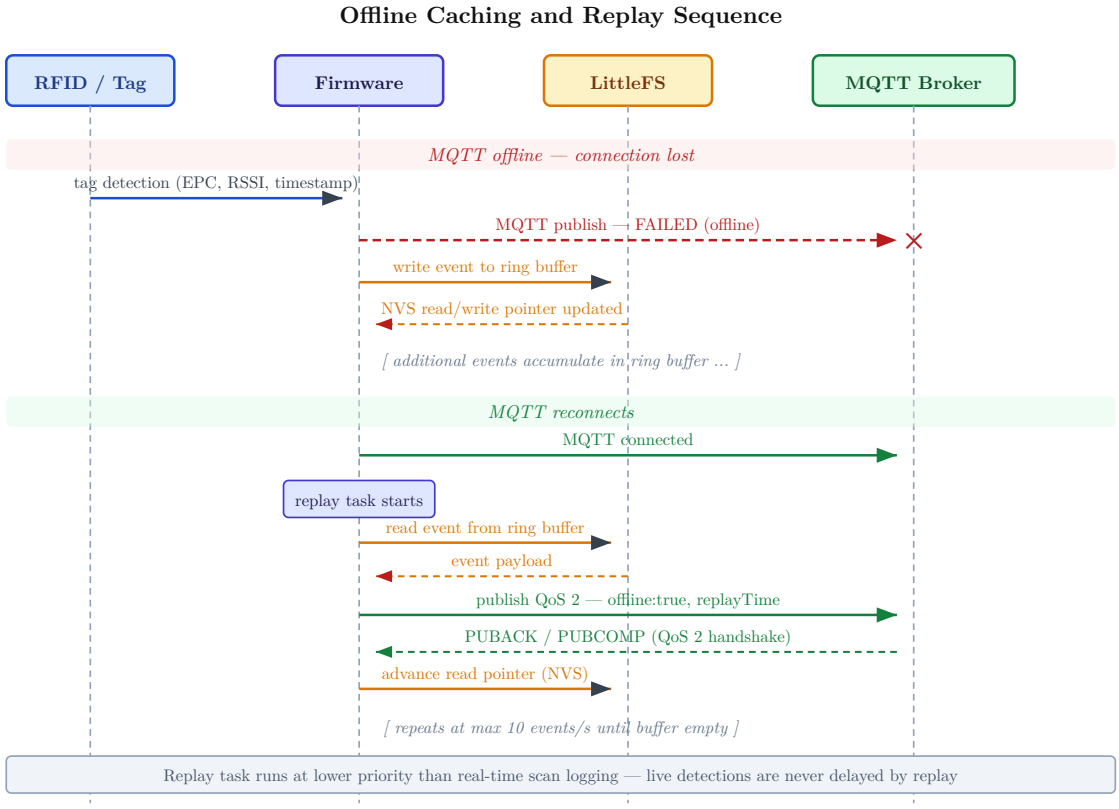

Figure 10: Offline caching and replay sequence diagram

3.3.5. User Interaction: Buttons, LEDs, and Gestures

The IO subsystem manages four status LEDs, two tactile buttons, and one passive infrared motion sensor. All GPIO configuration, button debouncing, LED state management, and scan mode logic are encapsulated in a dedicated `io_controller` module, keeping the main application file (`lighthouse.c`) limited to orchestration responsibilities and MQTT/offline event routing. This separation improves maintainability and allows the IO logic to be tested and modified independently of the higher-level application state machine.

Debounce filtering is applied in software using a simple time-threshold approach: a button state change is only registered if the GPIO level remains stable for at least 50 ms after the initial transition. This 50 ms debounce threshold effectively suppresses mechanical contact bounce without introducing perceptible delay in the user experience.

Scan Modes

The firmware operates in one of two scan modes, togglable via a 3-second hold of BUTTON1. The mode determines whether RFID scan bursts are triggered automatically by IR motion detection or manually by button press. Before any mode transition executes, the firmware unconditionally stops any active RFID inventory operation. This prevents mode transitions from leaving the RFID reader in an undefined operational state or consuming power unnecessarily.

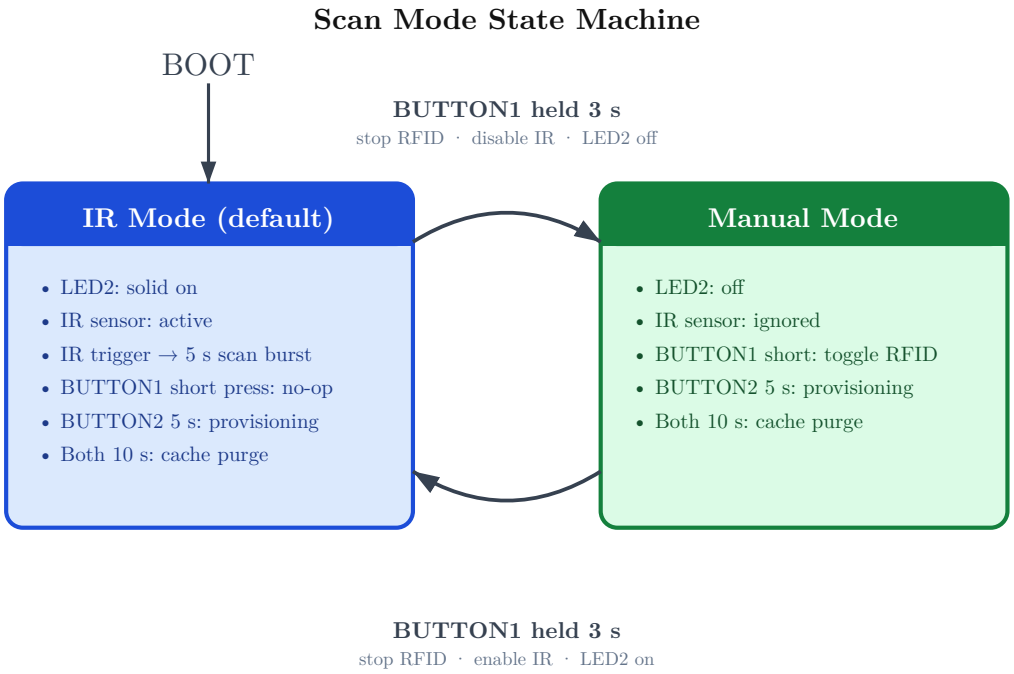

Figure 11: Scan mode state machine

Button Gestures

The firmware recognizes both short-press and long-hold gestures on each button, as well as a dual-button combo gesture for developer-level cache purge operations.

Table 11: Complete button gesture reference

Button	Gesture	Action
BUTTON1	Short press	Toggle RFID scan (Manual Mode only; no-op in IR Mode)
BUTTON1	3 s hold	Switch between IR Mode and Manual Mode
BUTTON2	Short press	Publish status/statistics message via MQTT
BUTTON2	5 s hold	Enter WiFi AP provisioning
Both	10 s hold	Cache purge: clear offline event cache

LED Indicator Behavior
Table 12: LED states during normal operation

LED	Condition	State
LED1 (green)	No WiFi connection	Off
LED1 (green)	WiFi connected, no MQTT	Blinking 1 s period
LED1 (green)	WiFi + MQTT connected	Solid on
LED2 (green)	IR Mode active	Solid on
LED2 (green)	Manual Mode active	Off
SCANNING_LED (red)	RFID inventory in progress	On
SCANNING_LED (red)	No active inventory	Off
ACTIVITY_LED (yellow)	Tag detected	Brief 100 ms flash

Table 13: LED states during AP provisioning

LED	Provisioning phase	State
LED1 (green)	AP active, WiFi test not yet passed	Blinking
LED1 (green)	WiFi test passed	Solid on
LED2 (green)	AP active, MQTT test not yet passed	Blinking
LED2 (green)	MQTT test passed	Solid on

Battery Status LED Patterns (ACTIVITY_LED, Battery Power Only)
When the device is running on battery power, ACTIVITY_LED provides continuous battery level indication via repeating pulse patterns with different cadences corresponding to charge percentage thresholds. See Table 14

Table 14: Battery level indication

Battery level	LED pattern
USB powered	Off (tag detection flashes only)
Battery $> 10\%$	Brief flash every 5 seconds
Battery 5–10%	Pulsing 500 ms on/off
Battery $< 5\%$	Rapid pulsing 200 ms on/off

3.4. Server

3.4.1. Infrastructure and Stack

The server component runs as a single BunJS process that consolidates all backend subsystems: the HTTP REST API, an embedded MQTT broker implementing the firmware transport path established in Section 2.4, a WebSocket gateway for real-time dashboard updates, and several background pollers responsible for event processing and external system integration. This monolithic-process architecture eliminates the operational complexity of managing multiple service processes while maintaining clear internal component boundaries through modular TypeScript code organization.

The technology choices for each subsystem, with per-component selection rationale, are listed in Table 15. This architecture delivers a single deployable artifact - one process,

one repository, one configuration surface - while preserving internal modularity through TypeScript's module system and Hono's middleware composition.

Table 15: Server component responsibilities and selection rationale

Component	Technology	Role
HTTP API	Hono	REST endpoints
Lightweight, TypeScript-native; type-safe routing without legacy framework overhead; co-located with the MQTT broker in a single process, eliminating inter-service communication.		
MQTT Broker	Aedes	Receives firmware scan/health messages
In-process embedding removes the external service dependency of a standalone broker; a single portal generates at most a few scan events per second, well within single-process throughput.		
Database	PSQL, Drizzle ORM	Persistent event and user storage
Relational model suits the grouped lighthouse and tag-assignment schema; Drizzle provides compile-time type-safe queries and manages schema migrations via Drizzle Kit.		
WebSocket	Bun WebSocket	Real-time push to dashboard clients
Native Bun implementation requires no additional dependency; push-based delivery means dashboard clients receive events immediately without polling the REST API.		
Navigo3 Poller	Custom interval	Periodic retry of unsynced events
Simple polling loop is sufficient given the low event frequency; avoids message queue infrastructure; 60 s retry interval balances delivery promptness against Navigo3 API load.		
Event Sweeper	Custom interval	Cluster detection & direction processing
Fixed 2 s poll decouples ingestion rate from processing; bounded per-cycle cluster cap of 50 prevents a backlog from starving other subsystems during high traversal density.		

3.4.2. Event Ingestion and Raw Scan Storage

The firmware publishes RFID tag detection data to the server via MQTT using the topic pattern `lighthouse/{id}/scans`, where `{id}` is the device's MAC address. As described in Section 3.3.2-B, the firmware batches tag detections into JSON arrays to avoid saturating the MQTT broker with individual per-tag publishes during high-density scan windows. Each array element represents a single tag detection and contains the following fields: `epc` (the tag's Electronic Product Code), `rssiDbm` (received signal strength in decibels-milliwatts), `timestampMs` (the detection time as a Unix timestamp in milliseconds), and `timeBasis` (a quality indicator with values `synced`, `estimated`, or `relative`, as defined by the firmware's time synchronization subsystem in Section 3.3.3).

The server-side scan handler validates each array element independently. Malformed elements-those missing required fields or containing invalid data types-are logged and discarded without aborting processing of the remaining valid elements in the batch. This

per-element validation strategy ensures that transient firmware bugs or data corruption in transit cannot cause entire scan batches to be lost. Valid scan entries are persisted to the `raw_scans` table immediately upon receipt.

The `raw_scans` table functions as an immutable audit log. Once inserted, scan records are never updated or deleted by normal system operation; they serve as the authoritative source of truth for all downstream event processing and forensic analysis. The table schema includes the following key fields:

Table 16: `raw_scans` table key fields

Field	Type	Description
`id`	`bigint`	Auto-increment primary key
`lighthouseId`	`int`	FK to lighthouses
`epc`	`varchar`	Tag EPC identifier
`rssiDbm`	`int (nullable)`	Signal strength in dBm
`timestamp`	`timestamptz`	Tag detection time (from firmware)
`timeBasis`	`enum`	`synced` / `estimated` / `relative`
`processedAt`	`timestamptz (nullable)`	Set when cluster is processed
`orphanedAt`	`timestamptz (nullable)`	Set when scan is orphaned

The `processedAt` and `orphanedAt` columns both default to `NULL` at insertion time, marking the scan as pending. The event sweeper (described in Section 3.4.3) queries for scans where both fields are `NULL`, clusters them by tag EPC and lighthouse group, and invokes direction detection algorithms. Scans that successfully contribute to a detected entry or exit event have their `processedAt` field updated to the current timestamp; scans that remain isolated beyond the configured activity timeout are marked as orphans by setting `orphanedAt`.

The `timeBasis` field is stored verbatim and is not interpreted during ingestion. The decision to accept or reject scans based on timestamp quality is deferred to the event sweeper. During cluster formation, scans with `timeBasis` other than `synced` are filtered out, as only SNTP-synchronized timestamps provide the temporal precision required for direction detection. Non-synced scans are immediately marked as orphans and excluded from event processing. This separation of concerns keeps the ingestion path simple and fast while concentrating time quality logic in a single downstream component.

Health telemetry-system uptime, free heap memory, WiFi signal strength, and RFID reader responsiveness-is published by the firmware to a separate MQTT topic, `lighthouse/{id}/health`. These messages are stored in a dedicated `lighthouse_health_snapshots` table but are not part of the event processing pipeline. Health data is used exclusively for operational monitoring and diagnostics via the dashboard.

The ingestion architecture is designed for append-only, high-throughput write patterns. Database writes occur synchronously in the MQTT message handler callback, leveraging PostgreSQL's write-ahead logging to ensure durability without blocking the broker's event loop. Upon successful insertion, the server broadcasts a real-time scan event to all

connected WebSocket clients, enabling live tag detection visualization in the dashboard without requiring client polling.

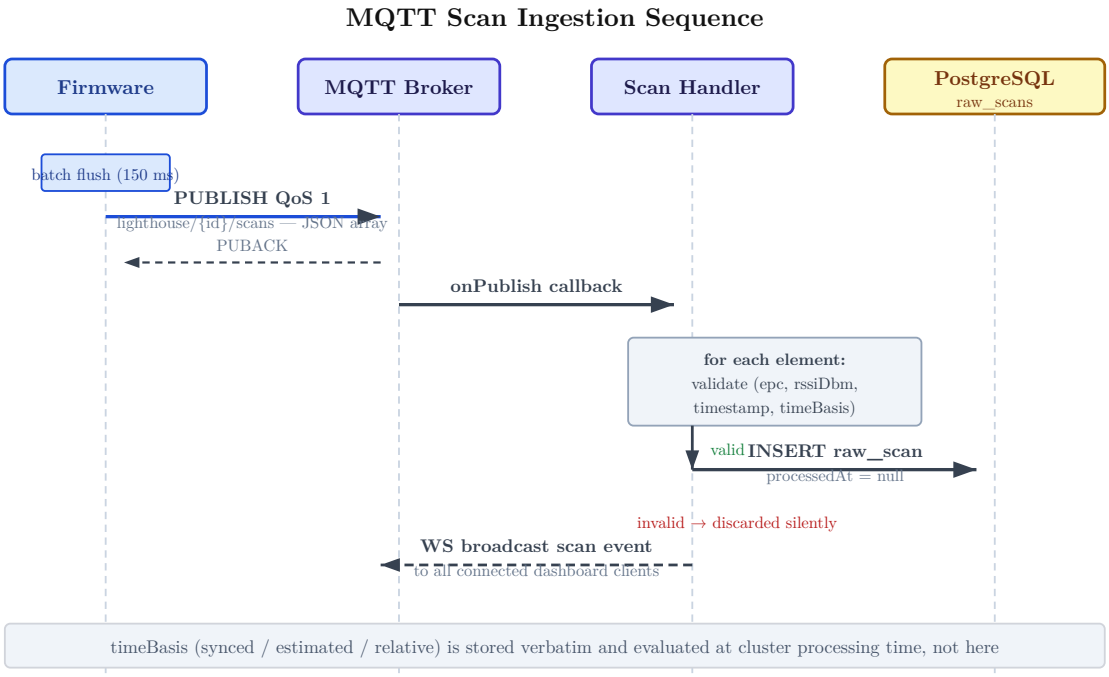

Figure 12: MQTT ingestion sequence

3.4.3. Direction Detection and Event Processing

Cluster detection and event sweeper

The event sweeper is a background polling service that identifies completed RFID scan clusters and invokes the direction detection pipeline. A cluster represents all raw scan records associated with a single tag traversal attempt through a portal-scans from both the inside and outside Lighthouse units, spanning the time interval during which the tag was within detection range of at least one unit. The sweeper runs at a fixed 2-second interval, decoupled from both the MQTT ingestion rate and the algorithmic processing time of individual clusters.

A cluster is defined as the set of all `raw_scans` records matching a specific `(epc, groupId)` tuple where the most recent scan timestamp is older than the group's configured `activityTimeoutMs` threshold. The default activity timeout is 4 seconds-this value establishes the maximum permissible gap between consecutive detections before the system considers the tag to have exited the detection zone. The timeout is stored per-group in the database and can be adjusted via the REST API to account for differences in walking speed, portal geometry, or reader sensitivity across deployment sites.

The cluster detection query operates on the `raw_scans` table, grouping unprocessed records (those with `processedAt IS NULL` and `orphanedAt IS NULL`) by `(epc, groupId)` and filtering the resulting groups with the condition `MAX(timestamp) < NOW() - activityTimeoutMs`. This temporal filter ensures that only clusters whose tag activity has conclusively ended are passed to the processing pipeline-scans from ongoing traversals remain pending until the activity timeout expires. The query additionally filters out

scans from Lighthouse units that have no assigned group, as these cannot participate in paired direction detection.

To bound the computational cost of each sweeper cycle and prevent a backlog of unprocessed clusters from causing unbounded processing delays, the sweeper processes a maximum of 50 clusters per tick. If more than 50 closed clusters exist at a given poll, the excess clusters are deferred to the next cycle. This cap ensures that the sweeper's execution time remains predictable and does not starve other server subsystems of CPU time during periods of high tag traversal density.

Scans from ungrouped Lighthouse units-those with `groupId IS NULL`-are handled separately. Such scans represent a system misconfiguration: a Lighthouse unit is publishing data but has not been assigned to a portal. These scans cannot contribute to direction detection and are orphaned immediately once they exceed a fixed 10-second timeout. The orphan reason is set to `misconfigured_group`, and the scans are excluded from all future processing. This separation prevents ungrouped units from polluting the event stream while preserving a record of the misconfiguration for diagnostic purposes.

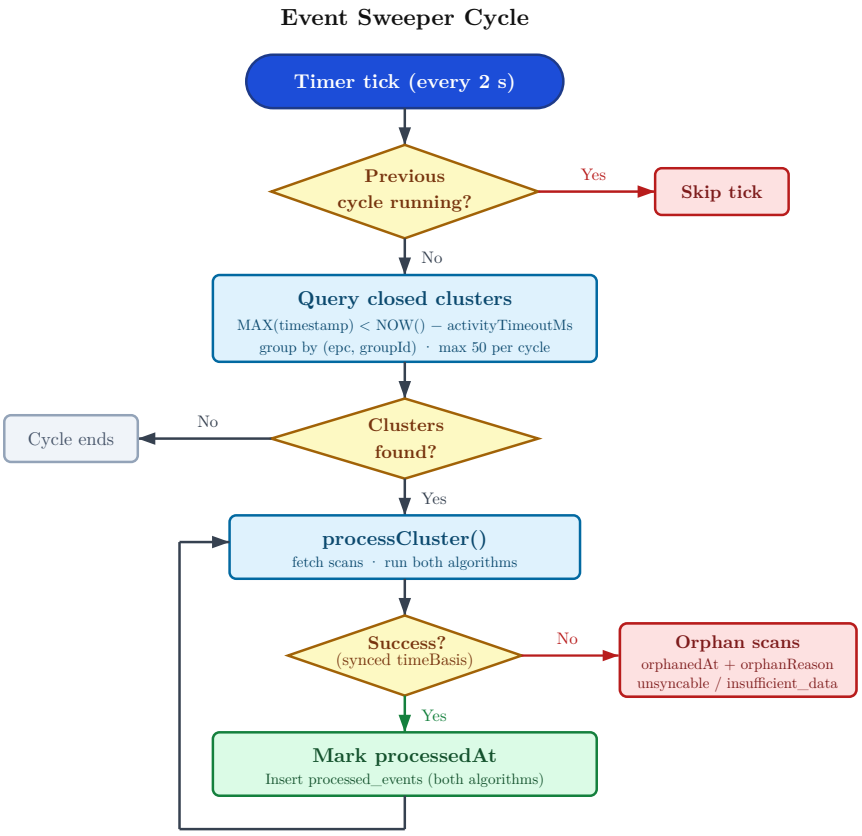

Figure 13: Event sweeper cycle flowchart

Algorithm 1 - Temporal Centroid (C_1)

Algorithm 1 implements the temporal centroid approach of Oikawa [6], surveyed in Section 2.2. For each Lighthouse group, the centroid is the arithmetic mean of its scan timestamps:

$$\bar{t}_{\text{out}} = \frac{1}{N_{\text{out}}} \sum_{i=1}^{N_{\text{out}}} t_i^{\text{out}}, \quad \bar{t}_{\text{in}} = \frac{1}{N_{\text{in}}} \sum_{i=1}^{N_{\text{in}}} t_i^{\text{in}}$$

The earlier centroid identifies the side the tag passed first: $\bar{t}_{\text{out}} < \bar{t}_{\text{in}}$ classifies the event as entry (IN), the reverse as exit (OUT). If the absolute difference is at most 1 ms - within timestamp quantisation - direction is marked unknown.

The confidence score for Algorithm 1 is the product of three dimensionless factors:

$$C_1 = \text{CSF} \times \text{CSzF} \times \text{BCF}$$

Each factor is clamped to the range $[\text{FLOOR}, 1.0]$ with $\text{FLOOR} = 0.1$; the clamp prevents any single degenerate factor from collapsing C_1 to zero.

Centroid separation factor (CSF):

$$\text{CSF} = \frac{|\bar{t}_{\text{out}} - \bar{t}_{\text{in}}|}{t_e - t_s}$$

where t_s and t_e are the earliest and latest scan timestamps in the cluster. CSF approaches 1.0 when the two scan groups are well-separated in time and falls toward zero when they overlap, which is common when a person moves slowly or pauses within the detection zone. If $t_e = t_s$ (all scans at the same timestamp), CSF is clamped to FLOOR.

Cluster size factor (CSzF):

$$\text{CSzF} = \min\left(1.0, \frac{n_{\text{total}} - 2}{8}\right)$$

CSzF rewards clusters with more total scans as stronger statistical evidence, saturating at 10 scans. The saturation prevents the score from biasing toward slow carriers or unusually responsive tags.

Bilateral coverage factor (BCF):

$$\text{BCF} = \frac{\min(n_{\text{in}}, n_{\text{out}})}{\max(n_{\text{in}}, n_{\text{out}})}$$

BCF measures the balance of scan counts between the two Lighthouses; it is 1.0 when both contribute equally and falls when one dominates, which can occur if the tag is carried along the extreme edge of the portal or if one unit's antenna has degraded range. Clusters with zero scans on one side are orphaned upstream as `insufficient_data` (Section 3.4.3-D) and never reach BCF.

Figure 14: Cluster timeline screenshot

Algorithm 2 - RSSI-Weighted Centroid (C_2)

Algorithm 2 follows the same temporal centroid structure as Algorithm 1 but weights each scan by a monotonically increasing function of its RSSI, $w_i = f(\text{RSSI}_i)$, drawing on the signal-strength direction cue of Jie et al. [7] surveyed in Section 2.2. The specific form of f is implementation-defined; the algorithm requires only that stronger signals yield higher weights. The weighted centroid for one Lighthouse group is:

$$\bar{t}_w = \frac{\sum_i w_i \cdot t_i}{\sum_i w_i}$$

Stronger signals - produced when the tag is closest to a reader's antenna - pull the effective centroid toward their timestamps, disambiguating cases where the arithmetic means of the two groups are nearly equal. Direction inference is otherwise identical to Algorithm 1, and CSzF and BCF are reused unchanged. CSF is reused with the weighted centroids $\bar{t}_{\text{w,out}}$ and $\bar{t}_{\text{w,in}}$ substituted for the arithmetic means in its numerator; this weighted variant is denoted CSF_w below.

Algorithm 2 additionally fits a least-squares regression line to RSSI versus time for each Lighthouse group. The outside group yields the slope m_{out} of the best-fit line through the points $(t_i^{\text{out}}, \text{RSSI}_i^{\text{out}})$, and similarly the inside group yields m_{in}. The agreement between these observed slopes and those expected for the inferred direction forms a fourth confidence factor:

RSSI trend consistency factor (RTCF):

RTCF is categorical: 1.0 when both slope signs match expectation (a falling slope at the outside reader and a rising slope at the inside reader for an entry; reversed for an exit), FLOOR when both contradict, and 0.5 when inconclusive - either side has fewer

than 3 scans, the coefficient of determination R^2 is below 0.1, or one side agrees while the other is inconclusive. The neutral 0.5 (rather than FLOOR) for inconclusive cases accommodates noisy but otherwise valid detections, where multipath or orientation-dependent variation degrades the linearity of the RSSI-versus-time relationship.

The composite confidence is:

$$C_2 = \mathrm{CSF}_w \times \mathrm{CSzF} \times \mathrm{BCF} \times \mathrm{RTCF}$$

where CSF_w denotes CSF computed from the weighted centroids.

Orphan handling
Not all scan clusters can be successfully processed into directional events. Three distinct failure modes exist, each resulting in scans being marked as orphans with a specific reason code stored in the `orphanReason` field of the `raw_scans` table.

The `insufficient_data` reason applies to clusters where only one Lighthouse in the group contributed scans. This can occur if a tag was detected by the outside reader but never entered the range of the inside reader, or vice versa. Such single-sided clusters lack the bilateral information required for direction detection and cannot produce a valid traversal event. However, these scans are not orphaned immediately upon cluster closure. Instead, they remain in the pending state until the cluster's age exceeds the group's `orphanTimeoutMs` threshold (distinct from the `activityTimeoutMs` used to determine cluster closure). The default orphan timeout is 8 seconds. This delayed orphaning allows for the possibility that additional scans from the missing Lighthouse might arrive late-due to firmware batching delays, MQTT retransmission, or offline cache replay-and complete the cluster retroactively. Only after the orphan timeout expires are the scans marked with `orphanedAt` and `orphanReason = insufficient_data`.

The `unsyncable` reason applies when all scans in a cluster have a `timeBasis` value other than `synced`. As described in Section 3.3.3, the firmware attaches a time quality indicator to every scan event. Scans with `timeBasis` of `estimated` or `relative` lack the absolute timestamp accuracy required for centroid calculations and are therefore ineligible for direction detection algorithms. If a cluster consists entirely of non-synced scans, those scans are orphaned immediately with the `unsyncable` reason. If a cluster contains a mix of synced and non-synced scans, only the non-synced scans are orphaned; the synced scans remain in the pending state and may pair with future scans from the same tag and group.

The `misconfigured_group` reason applies to scans from Lighthouse units that have no `groupId` assigned in the database. Such units are not part of any portal and therefore cannot participate in paired direction detection. These scans are orphaned immediately upon detection by the sweeper, once they exceed the fixed 10-second ungrouped timeout. The rapid orphaning of misconfigured scans prevents them from accumulating indefinitely in the pending state and provides a clear diagnostic signal that a Lighthouse unit requires administrative intervention to be assigned to a group.

3.4.4. User and Tag Management

The system maintains an internal user registry separate from the external enterprise system to allow for decoupled operation and to support deployment scenarios where

no external integration exists. Users are identified internally by a UUID primary key (`id`) that is never exposed to external systems. The `syncId` field stores the numeric user identifier from the external system-in this implementation, Navigo3-and serves as the foreign key for integration API calls. This dual-identifier design isolates the attendance system's data model from changes to external user ID formats or migration events where user IDs are reassigned in the enterprise system.

A user record can exist with or without a `syncId` value. Users created locally through the dashboard initially have `syncId` set to null and function as standalone entities within the Lighthouse system-their tag traversals are detected and logged, but no attempt is made to synchronize these events to Navigo3. When a `syncId` is later assigned via the dashboard or API, all past unsynced events for that user become eligible for retroactive integration push via the Navigo3 retry sweep. This design supports phased rollout scenarios where users are onboarded to the physical RFID system before their accounts are provisioned in the external software.

Each user can have multiple RFID tag EPCs assigned simultaneously. This accommodation reflects the practical reality that individuals may carry multiple tagged items-a lanyard-mounted tag, a second tag embedded in a wallet or bag, or temporary tags issued during badge replacement. Tag assignments are stored in a separate `tag_assignments` table with a many-to-one relationship to users. Each assignment includes an `assignedAt` timestamp and a nullable `deactivatedAt` timestamp. When a tag is reassigned to a different user, the previous assignment is soft-deleted by setting `deactivatedAt` to the current time rather than being removed from the database, preserving a complete audit trail of tag ownership history.

A database constraint enforces that at most one active assignment exists per tag EPC at any given time-two users cannot simultaneously claim the same tag. The uniqueness constraint is implemented as a partial index: `UNIQUE (tagEpc) WHERE deactivatedAt IS NULL`. This structure allows a tag to appear in the `tag_assignments` table multiple times across its lifecycle, with all historical assignments retained but only the most recent active assignment participating in user resolution during event processing.

3.4.5. Navigo3 Integration

The Navigo3 integration layer consumes the REST path established in Section 2.4 via the `dry-api` framework [13], [14]. It is implemented as an isolated service module that consumes processed events from the direction detection pipeline. The service is gated entirely on the presence of the `NAVIGO3_BASE_URL`, `NAVIGO3_USERNAME`, and `NAVIGO3_PASSWORD` environment variables at server startup; if any are absent the service initialises in a disabled state and imposes no runtime overhead on the event processing pipeline.

When enabled, the service authenticates against the Navigo3 API at startup via `POST /api/login`, exchanging username and password for a session token used as the bearer credential in all subsequent requests. The token remains valid until the server process terminates or Navigo3 invalidates the session. The numeric `typeId` for the `atWork` attendance type is queried once after authentication and cached in-process for the lifetime of the server.

A processed event is eligible for push when (1) its `algorithmId` matches `NAVIGO3_ALGORITHM_ID` (`temporal_centroid` or `rssi_weighted_centroid`), (2) its `direction` is `in` or `out` (`unknown` events are never pushed), and (3) the associated user has a non-null `syncId`. The `algorithmId` filter allows the operator to select which detection algorithm feeds the enterprise system while retaining both algorithms' outputs in the database for comparison (Section 3.4.3). Three Navigo3 endpoints are used, described in Table 17.

Table 17: Navigo3 endpoints used by the integration layer.

Endpoint	Direction	Purpose
`/api/attendance/embedded/start`	`in`	Register arrival
Invoked when an event with `direction = in` is pushed individually (no eligible counterpart present). Payload: `userId` (parsed from `syncId`), entry timestamp (ISO 8601), cached `typeId` for `atWork`, empty comment.		
`/api/attendance/embedded/stop`	`out`	Register departure
Invoked when an event with `direction = out` is pushed individually. Payload: `userId`, exit timestamp (ISO 8601), empty comment. `typeId` is omitted; Navigo3 infers it server-side from the most recent open attendance interval for the user.		
`/api/attendance/embedded/upsert`	`in + out`	Create or update closed interval
Invoked when a counterpart event (opposite direction, same `userId`, same calendar day, also unsynced) is found at push time; both events are sent together. Payload: `userId`, `day`, `timeFrom` and `timeTo`, `createdFrom` and `createdTo` (ISO 8601), `typeId`, empty comment, `changedBy = 0`. Idempotent - creates the interval if absent, updates timestamps if present. Returns the record's primary `id`, written to `navigo3RecordId` on both event rows. `changedBy` is a placeholder required by Navigo3′s schema validation; the server overwrites it with the authenticated session user.		

Immediate push

After the event processing transaction commits - both algorithm result rows written to `processed_events`, all constituent raw scans marked with `processedAt` - the event processor invokes `pushEvent()` asynchronously via `setImmediate()`, decoupling the integration call from the event sweeper's next cycle. `pushEvent()` selects between `start` and `stop` based on the event's direction (see Table 17). On HTTP 200 the `syncedToIntegration` flag on the `processed_events` row is set to `true`; on any failure (network, expired session, validation rejection) the flag remains `false` and the event is left to the retry sweep. Exceptions are not propagated out of the `setImmediate` block, isolating integration failures from the core attendance pipeline.

Paired upsert

Before invoking `pushEvent()`, the integration service checks `processed_events` for a counterpart - same `userId`, opposite direction, same calendar day, `syncedToIntegration = false`. If one is found, `pushPair()` is invoked instead, sending both events to the `upsert` endpoint (see Table 17) as a single closed attendance interval. On success the returned record `id` is written to `navigo3RecordId` on both event rows and both `syncedToIntegration` flags flip to `true` in a single database transaction, ensuring the pair

is never reprocessed even if the server restarts between the API call and the database update.

Retry sweep

The Navigo3 poller runs at a configurable interval (default 60 s, adjustable via `NAVIGO3_RETRY_INTERVAL_MS`) and re-attempts events that failed initial push. Each cycle the poller selects up to 100 events where `syncedToIntegration = false`, ordered by timestamp ascending, applying the same eligibility criteria as the immediate push path. For each event the poller first checks for a counterpart; if one is found and is also unsynced it invokes `pushPair()`, otherwise it falls back to `pushEvent()`. This two-tier strategy prefers sending complete intervals over isolated transitions.

Errors during retry are logged per event and do not abort the sweep. Failed events remain unsynced and are retried on subsequent cycles until they succeed or are manually resolved by an administrator. If a sweep is still in progress when the next interval elapses, the new cycle is skipped rather than running concurrently.

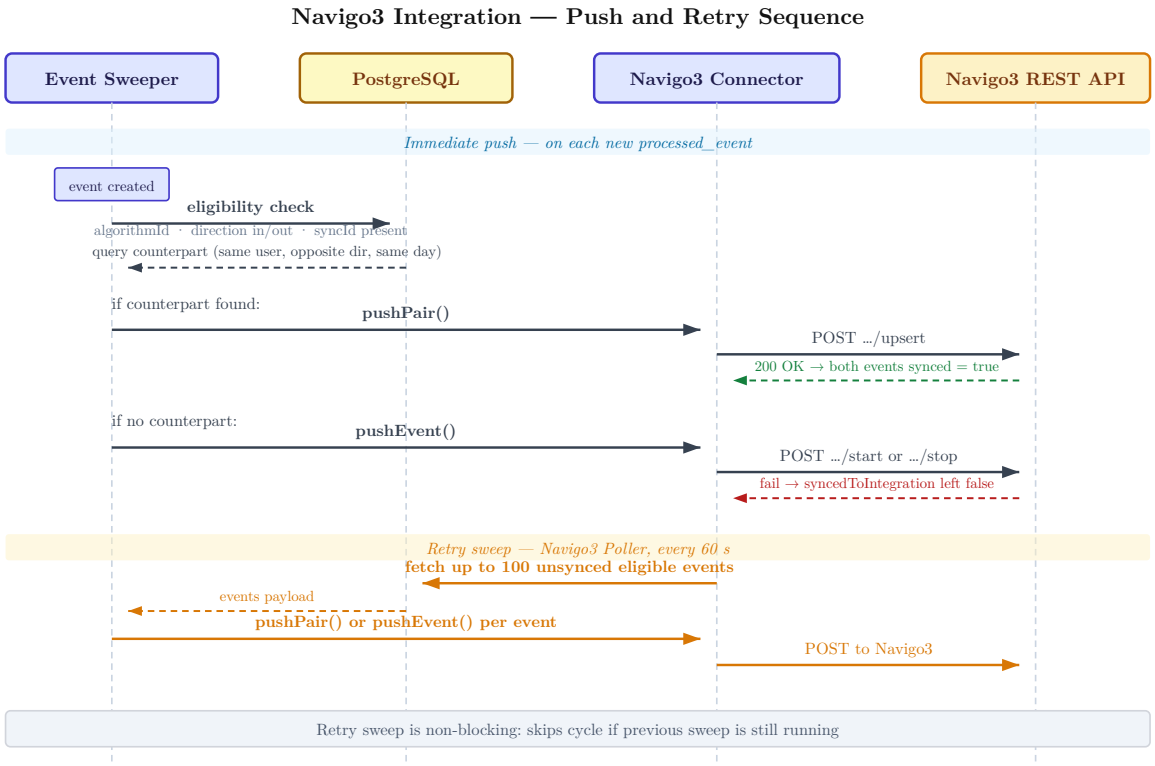

Figure 15: Navigo3 integration sequence

3.5. Dashboard

3.5.1. Architecture and Stack

The dashboard is a single-page application built with SolidJS, chosen for its fine-grained reactivity model: component state updates propagate directly to the DOM without a virtual-DOM diffing pass, keeping the runtime footprint small. Client-side routing is handled by `@solidjs/router` with five declared routes; the compiled static build is served directly from the BunJS process that hosts the REST API and MQTT broker, eliminating the need for a separate static file server. Per-component CSS modules provide style encapsulation.

Real-time updates are delivered over a single WebSocket connection opened on application mount in `App.tsx`. A central WebSocket store receives incoming messages and dispatches them to page-specific reactive stores, so only the relevant page re-renders on each incoming event; no polling is required. Table 18 lists the seven message types produced by the server and their consumer pages. Authentication is outside the scope of this prototype; the dashboard and REST API are accessible without credentials on the local network, with session management and role-based access control identified as future work.

Table 18: WebSocket message types and their consumer pages

Message type	Trigger	Consumer page(s)
`scan`	New raw scan ingested	Events
`device:online`	Lighthouse MQTT connect	Lighthouses
`device:offline`	Lighthouse MQTT disconnect / LWT	Lighthouses
`device:health`	Health telemetry received	Lighthouses
`device:pending`	Unknown device connects	Lighthouses
`event:new`	Processed event created	Processed Events
`event:orphaned`	Scans orphaned by sweeper	Processed Events

3.5.2. Pages

Lighthouses (`/`)
The root page lists all registered Lighthouse devices with their current status: online/offline indicator and the most recent health telemetry payload - uptime, WiFi RSSI, and RFID reader state. Health data arrives via `device:health` WebSocket messages and updates the display without a page refresh. Devices that connect to the MQTT broker but are not yet registered appear in a separate *pending* section; the operator assigns a name and a placement (inside/outside) to claim the device. The ESP32 eFuse MAC address serves as the stable device identifier throughout registration.

Groups (`/groups`)
A group pairs two Lighthouses - one designated outside, one inside - into a detection portal. The page manages the group label and description; the two per-group timing parameters (`activityTimeoutMs`, default 4 000 ms; `orphanTimeoutMs`, default 8 000 ms) are stored in the database and configurable via the REST API, but UI controls for these values are out of scope for this prototype. A Lighthouse may belong to at most one group; scans from ungrouped units are orphaned by the event sweeper and never produce processed events.

Events (`/events`)
The Events page is a live feed of raw scan records as they arrive from Lighthouse units. New scans are prepended to the table via the `scan` WebSocket message. The table is filterable by Lighthouse, EPC (partial match), scan source (`realtime` / `offline_sync`), and date range. The page serves as a diagnostic tool, allowing the operator to confirm that both units in a portal are detecting tags before trusting the direction detection output.

Processed Events (`/processed`)

This page displays the output of the direction detection pipeline. Each row represents one processed event and shows direction, confidence, algorithm, tag EPC, assigned user, group, and timestamp. Rows are filterable by algorithm, direction, user, group, and date range; both algorithms can be viewed simultaneously in a merged, time-sorted view. A notification banner appears when new `event:new` WebSocket messages arrive while the page is open, allowing the operator to refresh without leaving the page.

Each row opens a detail modal with three tabs:

- **Overview** - direction, confidence percentage, tag EPC, user, group, algorithm, timestamp, and cluster time span; confidence factor breakdown rendered as horizontal progress bars (centroid separation, cluster size, bilateral coverage, and - for Algorithm 2 - RSSI trend consistency); a link to the companion event produced by the other algorithm for direct comparison.
- **Raw Scans** - all raw scans belonging to the cluster, grouped by Lighthouse (inside vs. outside), with EPC, RSSI, timestamp, and time basis.
- **Timeline** - a scan-timing histogram with a horizontal time axis, bars coloured by Lighthouse (blue = outside, orange = inside), and dashed centroid markers. For Algorithm 2, an RSSI-over-time scatter plot with per-Lighthouse linear regression lines is rendered below the histogram, illustrating the signal trend used in the RSSI Trend Consistency Factor calculation.

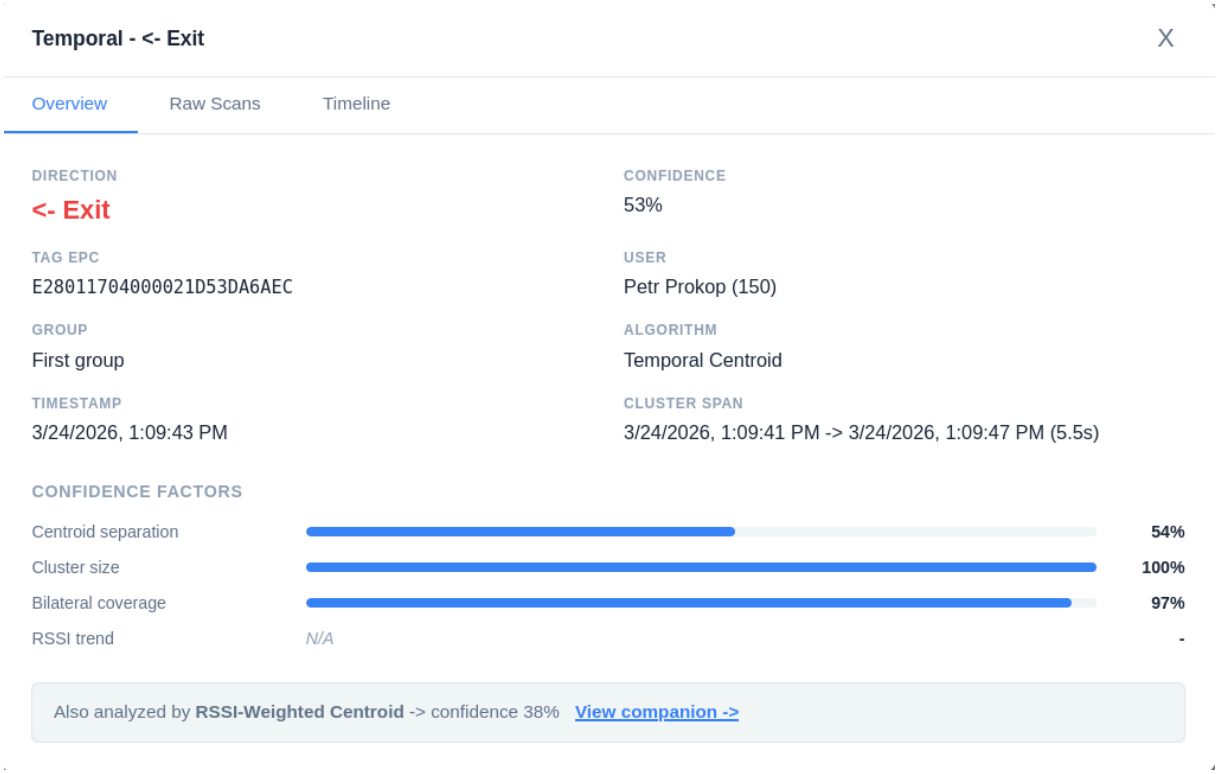

Figure 16: Processed event detail modal - Overview tab showing direction, confidence, confidence factor bars, and companion algorithm link.

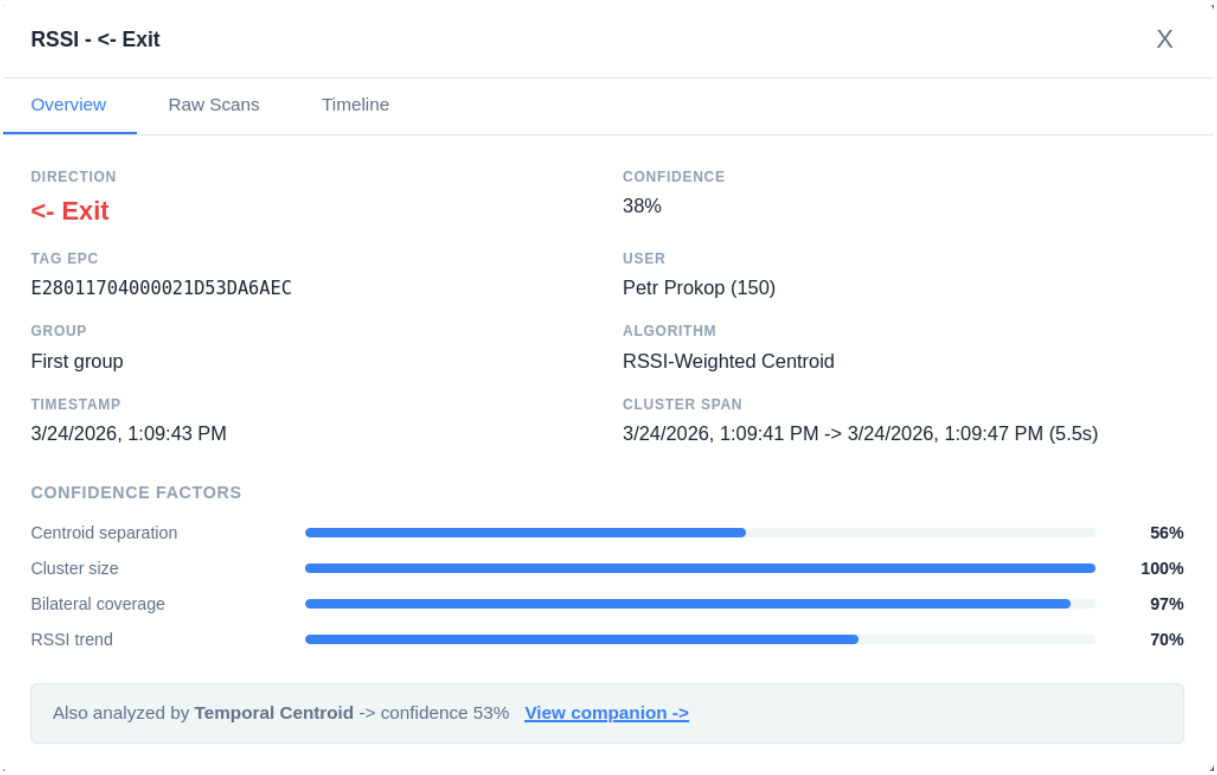

Figure 17: Processed event detail modal - Timeline tab showing scan timing histogram with centroid markers and RSSI scatter plot with regression lines.

Users (`/users`)
The Users page manages user records comprising a display name, email address, and a Navigo3 sync ID used by the integration layer. Each user may have multiple EPC tag assignments active simultaneously; deactivated assignments are retained with a `deactivatedAt` timestamp for audit purposes rather than being deleted.

3.6. System verification

All validation was conducted using three Board v2 Lighthouse units running production firmware, each assembled in its final 3D-printed enclosure. The detection portal for all multi-unit tests was formed by the Red unit (designated OUTSIDE) and the Yellow unit (designated INSIDE); both portal units are equipped with SMA receptacle antenna connections. The Blue unit participated only in the standalone detection range measurement. The server ran on a local area network with an embedded Aedes MQTT broker, PostgreSQL storage, and a Navigo3 test instance connected via the integration layer described in Section 3.4.5. The event sweeper was configured with `activityTimeoutMs = 4000 ms` and a polling interval of 2 seconds throughout all sessions.

3.6.1. Detection range

Per-unit detection range was measured with each unit in isolation. A passive UHF tag was held up oriented directly toward the antenna. Maximum reliable range was defined as the furthest distance at which the tag was detected in every one of three consecutive scan windows; distance was stepped in 0.5 m increments. Red was measured at two points in the test campaign due to a mechanical event discussed below.

Table 19: Detection range per unit

Unit	Antenna	Antenna connection	Max reliable range [m]	Condition
Red	5.5 dBi	SMA	2.5	Pre-drop
Red	5.5 dBi	SMA	1.5	Post-repair
Yellow	5.5 dBi	SMA	3.0	N/A
Blue	4 dBi	Direct-solder	3.0	N/A

Yellow and Blue achieve the same 3.0 m maximum reliable range despite Yellow carrying a higher-gain 5.5 dBi antenna. Under vendor-stated ideal conditions the 5.5 dBi antenna reaches approximately 1.3 m further than the 4 dBi antenna [17], [18]; the absence of any such advantage in the measured ranges is consistent with non-trivial forward-path loss on Yellow's feed, attributed in Section 3.2.3 to the non-impedance-controlled 2 cm microstrip trace. Red's pre-drop range of 2.5 m was already 0.5 m below Yellow's despite identical antenna and connection type, attributable to assembly-level variance in the SMA cable and connector.

3.6.2. Direction detection accuracy

Controlled traversals were performed through the portal at normal walking pace. Each traversal was logged as a processed event by the server; the algorithm's direction output was compared against the intended direction. Detection rate is reported as $\left(\frac{n_{\text{detected}}}{n_{\text{attempted}}}\right) \times 100\%$ with 95% Wilson score confidence intervals; missed detections were identified post-hoc from raw scan data as one-sided clusters orphaned by the sweeper as `insufficient_data` (per Section 3.4.3-D).

Table 20: Direction detection accuracy per algorithm

Algorithm	Correct [n]	Incorrect [n]	Accuracy [%]
Temporal Centroid (C_1)	78	0	100.0
RSSI-Weighted (C_2)	78	0	100.0

Direction accuracy across the full test campaign was 100% on both algorithms over 78 detected traversals. The detection rate, however, varied substantially with tag carry method. Under unobstructed hand-held conditions, 41 of 42 attempted traversals were detected (97.6%; 95% CI: 87.7-99.6%). With the tag carried in a near-side front trouser pocket at close range, 20 of 23 traversals were detected (87.0%; 95% CI: 67.9-95.5%). At longer range in the same pocket position, detection rate fell further (5 of 7; 71.4%; small sample). Tag carry on a lanyard presenting the tag edge-on to the antennas, and carry in a cross-body trouser pocket, produced effectively zero detection in both cases; these failure modes are a consequence of UHF tag polarisation physics interacting with the chosen portal geometry rather than a system limitation. The system's failure mode is exclusively non-detection: the `insufficient_data` orphan logic in the event sweeper guarantees that no direction call is produced from a one-sided cluster, so any event that reaches the database reflects bilateral evidence of traversal.

The bilateral coverage factor (BCF, defined in Section 3.4.3) quantifies the balance of scan counts between the two units per cluster. Its mean shifted from 0.59 under

unobstructed hand-held conditions to 0.42 in the close-range pocket session and 0.32 at longer range, reflecting the signal attenuation introduced by body absorption. Despite this degradation, every detected cluster produced a correct direction call, confirming that the temporal centroid approach is robust to scan asymmetry provided at least one scan is received from each unit.

Mean C_1 confidence across all sessions was 0.327 ± 0.155; mean C_2 confidence was 0.176 ± 0.099, a ratio of approximately 1.86:1. The C_2 deficit is driven by the RSSI Trend Consistency Factor, which rarely achieves high values during a normal walking traversal because the RSSI signal over a four-second window is noisy rather than monotonic. This does not affect directional accuracy; both algorithms produced identical directional outputs on every detected traversal.

3.6.3. End-to-end processing time

End-to-end latency was measured as the interval between `cluster_started_at` - the timestamp of the first RFID scan in a cluster - and the Navigo3 audit log database commit time for the resulting attendance record. This interval captures the full server-side pipeline: scan accumulation, `activityTimeoutMs` expiry, sweeper polling delay, direction detection, HTTP POST to Navigo3, and database write. The `cluster_started_at` timestamp lags the physical IR trigger by approximately 400 ms; true end-to-end latency from IR trigger is therefore approximately 400 ms greater than the values in Table 21. The dataset covers 15 events from 8 attendance records (8 IN and 7 OUT events).

Table 21: End-to-end processing time (IR trigger → Navigo3 record)

Metric	**Value** [s]
Theoretical minimum	~11
Mean measured (cluster start → Navigo3)	9.54 ± 1.16
Mean measured (IR trigger → Navigo3, adjusted)	~9.94
Min measured	7.46
Max measured	11.69

All 15 events fell within the theoretical pipeline budget of approximately 11 seconds. The single event that marginally exceeded the budget (11.69 s) is attributable to a worst-case combination of cluster scan distribution and sweeper polling alignment. OUT events completed approximately 1.3 s faster than IN events on average. The standard deviation of 1.16 s across all events is consistent with the 0–2 s uniform jitter introduced by the sweeper's fixed polling interval, which is the dominant source of latency variance in this pipeline.

3.6.4. Offline replay verification

Between the testing sessions the Red unit was dropped and subsequently repaired; a post-repair range measurement confirmed the reduction from 2.5 m to 1.5 m recorded in Table 19, which resulted in a scan count asymmetry favouring the Yellow unit during the offline replay session that follows.

Twelve alternating traversals were performed during a deliberate offline window of approximately four minutes and twenty-five seconds. The server was then restarted and replay was observed to completion.

All 606 replayed scans carried `timeBasis = synced`, confirming that SNTP wall-clock timestamps were preserved correctly through the LittleFS ring buffer. The sweeper successfully clustered every traversal's scans bilaterally despite the replay rate of approximately ten entries per second, producing 12 processed events in perfect alternating OUT/IN sequence. No stale entries from prior sessions were re-delivered. Confidence and BCF distributions were consistent with the hand-held benchmark session, confirming that no degradation in clustering quality results from the replay path relative to live operation.

3.6.5. Algorithm comparison

The combined dataset of 78 detected traversals, spanning the hand-held benchmark, near- and far-side pocket sessions, and the offline replay session, was used for a head-to-head comparison of Algorithm 1 (Temporal Centroid, C_1) and Algorithm 2 (RSSI-Weighted Centroid, C_2). Both algorithms are run on every cluster independently by the event sweeper per Section 3.4.3 the comparison is therefore a post-hoc analysis of stored outputs requiring no reprocessing.

Table 22: Algorithm comparison summary (combined dataset, n = 78)

Metric	Temporal Centroid	RSSI-Weighted Centroid
Overall accuracy [%]	100.0	100.0
Correct directions [n]	78	78
Incorrect directions [n]	0	0
Mean confidence (all calls)	0.327 ± 0.155	0.176 ± 0.099

Both algorithms classified direction correctly in every detected case under all tested conditions. The directional outputs of the two algorithms are perfectly correlated across the entire campaign. C_2's lower confidence reflects the additional gating effect of the RSSI Trend Consistency Factor: when RSSI is noisy over the cluster window - which is the common case during a normal walking traversal - the composite C_2 confidence is penalised even though the directional inference is correct. C_1's score, computed from centroid separation, cluster size, and bilateral coverage alone, is less susceptible to this penalty and produces confidence values that more directly reflect the geometric quality of the cluster.

C_1 is the recommended algorithm for production deployment. It produces identical directional outputs to C_2 with consistently higher and more interpretable confidence scores, and its factors map directly to measurable cluster properties that operators can reason about. C_2 remains valuable as an independent confirmation channel - agreement between the two algorithms on every processed event strengthens the evidence behind each attendance record - but does not provide additional discriminating power beyond C_1 under the conditions tested. Future work involving tag carrier identification or multi-tag clutter rejection may benefit from C_2's RSSI weighting in scenarios where C_1's temporal information alone is insufficient.

4. Conclusion

A passive, zero-interaction UHF RFID attendance system was designed and implemented in full. The system architecture, hardware design, firmware, server pipeline, and enterprise integration layer are documented in Chapter 3. Tags carried passively by employees are detected at normal walking pace without any deliberate action from the carrier, satisfying the zero-interaction requirement that motivated the project. The Navigo3 integration layer is isolated behind a connector interface, leaving the core detection pipeline independent of any particular enterprise platform.

The identification technology landscape was surveyed in Section 2, establishing UHF RFID operating at 860-960 MHz as the only commercially mature technology capable of passive, hands-free detection at 2-3 m range. Building on this finding, the system was architected as a portal model in which two Lighthouse units mounted on opposite sides of a doorway jointly supply raw RFID readings to a server-side direction detection pipeline, as documented in Chapter 3. Three Board v2 prototype units were fabricated, assembled, and brought to fully operational status, with the complete firmware and server pipeline implemented. End-to-end functionality was subsequently verified in a model office environment, addressing all four goals specified in the thesis assignment.

Laboratory validation confirmed that a passive tag traversal through the portal produces a correctly directed attendance record in Navigo3. Detection range was measured at up to 3.0 m for the intact units under controlled conditions. Both direction detection algorithms - the Temporal Centroid (C_1, Section 3.4.3-B) and the RSSI-Weighted Centroid (C_2, Section 3.4.3-C) - achieved 100% directional accuracy across a combined dataset of 78 detected traversals, with full inter-algorithm agreement on every event. C_1 is recommended for production deployment on the basis of its consistently higher and more interpretable confidence scores; the comparison and recommendation are detailed in Section 3.6. The offline resilience mechanism was verified during a deliberate network outage session in which all 606 buffered scan records were replayed without loss and correctly processed into attendance events. End-to-end latency from IR trigger to Navigo3 record averaged approximately 9.94 s across the test campaign, within the theoretical pipeline budget.

Testing in the model entrance corridor identified conditions under which the system does not operate reliably enough for practical deployment. The current prototype is not considered ready for production use and is best characterised as a laboratory-validated proof of concept establishing the technical feasibility of the approach.

Two structural limitations of the current hardware design are noted. The ESP32-WROOM-32 does not incorporate a hardware real-time clock; timekeeping relies on SNTP synchronisation, and timestamp quality degrades during prolonged network outages. This is mitigated in the current firmware by the `timeBasis` metadata field and NVS-persisted last-known time, but it remains a fundamental platform constraint. Each unit also carries a single antenna with no spatial diversity, making detection reliability sensitive to antenna placement and the integrity of the RF feed path; the coaxial joint between the YPD-R300 RF output pad and the antenna connector is a demonstrated failure point.

Several directions for future development are identified. A hardware real-time clock should be incorporated in a future board revision to eliminate timestamp degradation during offline periods. The SMA receptacle should be repositioned adjacent to the YPD-R300 RF output pad on the same revision to eliminate the intervening PCB trace and its associated impedance mismatch. Extended field testing over a full working day under real employee traffic is required before any deployment decision can be made. Better fail-safes and handling of edge-cases identified during development and testing represent the primary software and firmware development priority, alongside improved API security and session management in the operator dashboard.

5. Bibliography

[1] Navigo Solutions s.r.o., "Navigo3 – Software for Project and Company Management." 2026.

[2] "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)." [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[3] "ISO/IEC 14443: Cards and Security Devices for Personal Identification — Contactless Proximity Objects," no. ISO/IEC14443.

[4] "ISO/IEC 15693: Cards and Security Devices for Personal Identification — Contactless Vicinity Objects," no. ISO/IEC15693.

[5] "ISO/IEC 18000-63:2021: Information Technology — Radio Frequency Identification for Item Management — Part 63: Parameters for Air Interface Communications at 860~MHz to 960~MHz Type~C," no. ISO/IEC18000–63:2021. 2021.

[6] Y. Oikawa, "Tag Movement Direction Estimation Methods in an RFID Gate System," in *Proceedings of the 6th International Symposium on Wireless Communication Systems (ISWCS 2009)*, IEEE, 2009, pp. 41–45. doi: 10.1109/ISWCS.2009.5285332.

[7] Z. Jie and others, "RF-Access: Barrier-Free Access Control Systems with UHF RFID," *Applied Sciences*, vol. 12, no. 22, p. 11592, 2022, doi: 10.3390/app122211592.

[8] Espressif Systems, "ESP32 Technical Reference Manual." [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[9] M. Malý, *ESP32 prakticky: od základních obvodů k pokročilým aplikacím*, 1st ed. Praha: CZ.NIC, z.s.p.o., 2024.

[10] Espressif Systems, "ESP-IDF Programming Guide." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/

[11] D. L. Mills, "Simple Network Time Protocol (SNTP) Version 4 for IPv4, IPv6 and OSI," technical report RFC4330, Jan. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4330

[12] "MQTT Version 3.1.1." [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[13] Navigo Solutions s.r.o., "dry-api — Typed JSON-over-HTTP API Framework." [Online]. Available: https://github.com/NavigoSolutions/dry-api

[14] Navigo Solutions s.r.o., "API a předchystané integrace." [Online]. Available: https://www.navigo3.com/cs/api-a-predchystane-integrace

[15] Real Time Engineers Ltd. and Amazon Web Services, "FreeRTOS Kernel." [Online]. Available: https://www.freertos.org/Documentation/

[16] C. Haster, “littlefs — A Little Fail-Safe Filesystem Designed for Microcontrollers.” [Online]. Available: https://github.com/littlefs-project/littlefs

[17] “4 dBi UHF RFID circularly polarised panel antenna – product listing.” [Online]. Available: https://www.aliexpress.com/item/1005009293695195.html

[18] “5.5 dBi UHF RFID circularly polarised panel antenna – product listing.” [Online]. Available: https://www.aliexpress.com/item/1005009293695195.html

6. Appendices

A Use of generative AI

In accordance with the guidelines of Brno University of Technology, this appendix declares the use of generative AI in the preparation of this thesis.

Tools and period

Two tools from Anthropic were used between December 2025 and May 2026:

- **Claude** (claude.ai) - used as a conversational assistant for brainstorming, planning, drafting, and revision.
- **Claude Code** - used as an agentic coding tool, executing structured task documents I authored against the project codebase.

Scope and purpose

The tools were used in three distinct areas.

Architecture brainstorming and task authoring. Claude served as a discussion partner during the architectural design of the system. Conversations were used to test ideas, surface design alternatives, and pressure-test interface boundaries. The architectural decisions presented in this work are my own; the AI was used to challenge and refine them, not to originate them. The output of these sessions was a set of structured task documents preserved in the `tasks/` directory of the project repository, which served as the executable specifications for the subsequent implementation work.

Code implementation. Claude Code was used to implement parts of the codebase, following the task documents described above. For the firmware (ESP-IDF / C), implementations were produced from my task documents and the relevant protocol specifications and datasheets; the generated code was compiled and exercised on the physical Lighthouse units, and I made manual contributions to the redesign of selected internal subsystems and to the consolidation of redundant subsystems under shared abstractions. For the server (TypeScript / Bun), the public interfaces between subsystems were written by me, and the agent produced the implementations against those interfaces; the generated code was tested via the project's test suite and reviewed before being accepted into the codebase.

Thesis prose. Claude was used to draft and revise sections of the thesis text. All prose was read and edited by me. Where I identified inaccuracies or weak claims, I revised the text directly and used the chat to retrieve the relevant project context against which the claim could be verified.

Areas in which no AI tools were used

No generative AI tools were used for the electronics design, PCB layout, CAD work, or hardware manufacture of the Lighthouse units.

Verification and responsibility

All generated source code was tested - on the physical hardware for firmware, and through the server's automated test suite for server code - and refactored where required

before being accepted into the codebase. All thesis prose was read and edited by me, with technical claims anchored against primary sources cited throughout this work. I retain full responsibility for the content, correctness, and originality of this thesis.

Evidence in the project repository

The project repository is made publicly available on GitHub and is cited throughout this thesis. The artefacts of the AI-assisted work are preserved in the following directories:

- `tasks/` - structured task documents created as the output from the brainstorming sessions. These were used as input to Claude Code, containing the scope, file boundaries, protocol references, and acceptance criteria for each implementation task.
- `logbook/DEVLOG_*` - development logs documenting the progress, decisions, and outcomes of individual development sessions. These logs were generated by Claude Code after each session by letting the tool look through the changes in the Git repository with references to the implemented tasks. They were produced to create a knowledge base for the tools to then work from and source from when it came to writing the thesis.

These directories serve as the working log of the project and constitute the evidence trail for the AI-assisted portions of the work.

B Navigo3 Attendance API Reference

The Lighthouse integration layer communicates with Navigo3 through several API methods in the `attendance/embedded` endpoint group of the service's API. This appendix documents the four endpoints consumed by the integration layer. All endpoints require a valid authenticated Navigo3 session as an API type user with the mentioned methods whitelisted. The base URL is configured via the `NAVIGO3_BASE_URL` environment variable; all paths are relative to `/api/`.

Field types follow the Navigo3 schema convention: `string` maps to a JSON string, `number` to a JSON number, `datetime` to a string in the format `yyyy-MM-dd HH:mm:ss` (UTC), `date` to `yyyy-MM-dd`, and `time` to `HH:mm:ss`. Fields marked as optional may be omitted from the request payload entirely.

`attendance/embedded/start`

Registers the start of a work interval for a user. Called by `pushEvent()` when an eligible processed event with `direction = in` is pushed individually.

Table 23: `attendance/embedded/start` input fields

Field	Type	Required	Description
`userId`	`number`	Yes	Navigo3 user identifier. Parsed from the user's `syncId` field in the Lighthouse database.
`time`	`datetime`	No	Event timestamp. If omitted, Navigo3 records the server-side receipt time. The Lighthouse connector always supplies this field from the processed event's timestamp.
`typeId`	`number`	Yes	Attendance type identifier. Resolved at service initialisation via `attendance/embedded/types` by matching `systemName === "atWork"` and cached for the lifetime of the process.
`comment`	`string`	Yes	Free-text comment. The connector sends an empty string.

Output: no payload (`VoidParam`).

A successful response indicates the interval was opened in Navigo3. The connector sets `syncedToIntegration = true` on the corresponding `processed_events` row.

`attendance/embedded/stop`

Registers the end of a work interval for a user. Called by `pushEvent()` when an eligible processed event with `direction = out` is pushed individually.

Table 24: `attendance/embedded/stop` input fields

Field	Type	Required	Description
`userId`	`number`	No	Navigo3 user identifier. The connector always supplies this field.
`time`	`datetime`	No	Event timestamp. The connector always supplies this field.
`comment`	`string`	Yes	Free-text comment. The connector sends an empty string.

Output: no payload (`VoidParam`).

The `typeId` field is intentionally omitted; Navigo3 infers the attendance type from the most recently opened interval for the user. The connector sets `syncedToIntegration = true` on the corresponding `processed_events` row.

`attendance/embedded/upsert`

Creates or updates a closed attendance interval. Called by `pushPair()` when a counterpart event - opposite direction, same `userId`, same calendar day, also unsynced - is found at push time. The endpoint is idempotent: if a record for the given user and day already exists, Navigo3 updates its timestamps; otherwise it creates a new record.

Table 25: `attendance/embedded/upsert` input fields

Field	Type	Required	Description
`userId`	`number`	Yes	Navigo3 user identifier.
`day`	`date`	Yes	Calendar date of the interval, derived from the entry event timestamp.
`timeFrom`	`time`	Yes	Interval start time (UTC), derived from the entry event timestamp.
`timeTo`	`time`	Yes	Interval end time (UTC), derived from the exit event timestamp.
`createdFrom`	`datetime`	Yes	Full entry timestamp (UTC). Retained for audit purposes.
`createdTo`	`datetime`	Yes	Full exit timestamp (UTC). Retained for audit purposes.
`typeId`	`number`	Yes	Attendance type identifier, resolved as described under `start`.
`comment`	`string`	Yes	Free-text comment. The connector sends an empty string.
`changedBy`	`number`	Yes	Nominally the ID of the user making the change. The connector sends `0`; Navigo3 overwrites this server-side with the authenticated session user. The field is required by Navigo3 schema validation and cannot be omitted.
`id`	`number`	Yes	Record identifier for update operations. The connector sends `0` for new records; Navigo3 treats `0` as a creation request.

Output: `{ id: number }` - the primary key of the created or updated attendance record. The connector writes this value to the `navigo3RecordId` column on both the entry and exit `processed_events` rows, then sets `syncedToIntegration = true` on both in a single database transaction.

`attendance/embedded/types`

Lists all attendance types configured in the Navigo3 instance. Called once at service initialisation; not called again during normal operation.

Input: no payload (`VoidParam`).

Table 26: `attendance/embedded/types` output fields (per array entry)

Field	Type	Description
`id`	`number`	Numeric identifier used as `typeId` in `start` and `upsert` requests.
`name`	`string`	Human-readable display name of the attendance type.
`systemName`	`string`	Machine-readable identifier. The connector searches for the entry where `systemName == "atWork"`; if not found, the integration layer is disabled.

C Board Schematics

The schematic is organised as four hierarchical sheets produced in KiCad E.D.A. 9.0.9. Each sheet is self-contained and carries the full title block; cross-sheet connections are made via named net labels. See at ./attachments/LH-ELE.pdf

Table 27: Schematic sheet index

Sheet	Source file	Contents
LH-ELE-10	`lighthouse.kicad_sch`	Top-level sheet. ESP32-WROOM-32, TS1117B 3.3 V regulator, USB-C power input, DPDT power switch, fuse protection, status LEDs, push-buttons, BC337 low-side switch, battery monitoring, and all inter-sheet port connections.
LH-ELE-11	`charger.kicad_sch`	Charger submodule. TP4056 lithium-ion charge controller, DW01A protection IC, and FS8205A dual MOSFET.
LH-ELE-12	`setp_up_dc_dc.kicad_sch`	Step-up DC/DC converter submodule. SX1308 boost converter, SS24A Schottky diode, and adjustable feedback divider.
LH-ELE-13	`r300.kicad_sch`	UHF RFID reader submodule. YPD-R300 module connections, decoupling capacitance, EN line, and on-board buzzer circuit.

Four test jumpers are present on the board and are referenced in the schematics but are not visible in the PCB assembly drawing (Section), as they are located on the bottom side of the board. They were used during board bring-up to isolate and independently power each subsystem. Their default states and functions are as follows.

Table 28: Test jumper reference - bottom side of PCB

Ref	Default	Function
`JP1`	Closed	5 V rail to R300 subsystem; open to supply externally via TP7.
`JP2`	Closed	3.3 V rail to ESP32; open to supply externally via TP4.
`JP3`	Closed	Emitter return path of Q1 (BC337 low-side switch); open de-energises R300 regardless of GPIO5 state.
`JP4`	Open	Direct GND bypass for R300, circumventing Q1 entirely. Mutually exclusive with JP3.

PCB Assembly Drawing

The assembly drawing shows the top-side component placement for Board v2, produced from the KiCad PCB file. The drawing includes component courtyard outlines, reference designators, board outline with overall dimensions (83 × 80 mm), mounting hole positions, antenna keep-out zone, and board thickness (1.57 mm). Bottom-side features are limited to the four test jumpers documented in Appendix C.

See at ./attachments/LH-PCB-10.pdf

D Enclosure drawing

The enclosure drawing package consists of six sheets produced in FreeCAD 1.1.0. All parts are designed for additive manufacture (FDM); the general dimensional tolerance is ±0.3 mm unless otherwise specified on the individual drawing. Holes and mating features are noted for reaming or drilling to final size.

Table 29: Enclosure drawing sheet index

Sheet	Drawing number	File
Lighthouse - Assembly	LH-ASM-10	LH-ASM-10_assembly.pdf
Lighthouse - Exploded view	LH-ASM-20	LH-ASM-20_exploded.pdf
Lighthouse - Front face	LH-MEC-10	LH-MEC-10_front_face.pdf
Lighthouse - Back cover	LH-MEC-20	LH-MEC-20_back_cover.pdf
Lighthouse - PCB mount	LH-MEC-30	LH-MEC-30_pcb_mount.pdf
Lighthouse - Switch shim	LH-MEC-40	LH-MEC-40_switch_shim.pdf